

Mastering Enums in Java & Python

A Problem-Driven Workbook from Fundamentals to Advanced Design

A Professional Exercise Workbook with 60 Progressive Problems,
Cheat Sheets, and a Capstone Project

Programming Workbook Series

First Edition

Preface

Enums are one of the most underused and misunderstood features in both Java and Python. Most developers treat them as glorified integer constants. In reality, Java enums are full classes with constructors, fields, methods, and even per-constant behavior, while Python enums form a small but powerful type-safety layer with functional and flag-based variants that have no direct Java equivalent.

This workbook is built around a single idea: you learn enums by using them to solve real problems, not by reading a list of syntax rules. Every chapter introduces a small set of new enum capabilities and immediately puts them to work in problems ranging from a traffic light simulator to a red-black tree, a maze solver, and a turn-based strategy engine.

By the end of this workbook you will be able to choose, without hesitation, between a plain enum, a constant-specific method, an `EnumMap`, a Python `Flag`, or a `StrEnum` – and explain why that choice is correct in an interview or in production code.

Prerequisites: working knowledge of Java, Python, object-oriented programming, collections, and basic data structures and algorithms.

How to Use This Workbook

- Work through the phases in order. Each phase assumes mastery of the previous one.
- Read the problem statement fully before looking at the function signatures.
- Implement every exercise in *both* Java and Python. The comparison is the point.
- Do not skip the “Requirements” section – it enforces the enum feature being taught, not just any working solution.
- Attempt every exercise before reading its Hints.
- Solutions are intentionally not provided. Verify your own logic, then move to the Follow-ups to deepen the exercise.
- After each phase, complete the Mini Review before continuing.
- Track your progress using the Progress Tracker in the final chapters.

Symbol	Meaning
Easy	Core syntax and direct application
Medium	Combines multiple enum features
Hard	Requires design decisions and edge-case handling

Contents

Preface	1
How to Use This Workbook	2
Java Enum Cheat Sheet	5
Python Enum Cheat Sheet	7
Java vs Python Enum Comparison	9
1 Phase 1 – Basic Enums	10
Q1: Traffic Signal	10
Q2: Days of Week	11
Q3: Compass Direction	12
Q4: Chess Piece Color	14
Q5: Seasons	15
Q6: Count Enum Members	16
Phase 1 Summary	17
2 Phase 2 – Built-in Enum Methods	19
Q7: Enum Menu	19
Q8: Command Parser	20
Q9: Invalid Command	21
Q10: Compare Priorities	23
Q11: Month Number	24
Q12: Equality Test	25
Q13: Override toString / __str__	26
Q14: Case-Insensitive Parser	28
Phase 2 Summary	29
3 Phase 3 – Fields & Constructors	30
Q15: Planet Gravity	30
Q16: HTTP Status	31
Q17: Currency	33
Q18: Coin Calculator	35
Q19: Employee Roles	36
Q20: File Types	38
Q21: Countries	40
Q22: Maze Directions (dx, dy)	41
Phase 3 Summary	43

4	Phase 4 – Enum Methods	44
	Q23: Calculator	44
	Q24: Roman Numerals	45
	Q25: Direction Rotation	47
	Q26: File Extension Checker	48
	Q27: Temperature Converter	50
	Q28: Discount Calculator	51
	Phase 4 Summary	53
5	Phase 5 – Constant Specific Behavior	54
	Q29: Animal Sounds	54
	Q30: Tax Calculator	55
	Q31: Chess Piece Movement	57
	Q32: Compression Algorithms	59
	Q33: Notification Channels	60
	Q34: Payment Methods	61
	Phase 5 Summary	63
6	Phase 6 – Interfaces & Abstract Behavior	64
	Q35: Shape Interface	64
	Q36: Vehicle Interface	66
	Q37: Payment Interface	67
	Q38: Logger	69
	Q39: Database Connectors	70
	Phase 6 Summary	72
7	Final Remarks	73
	Final Remarks	73
A	Appendix: Solution Outline for Q1	79

Java Enum Cheat Sheet

```
public enum Signal {
    RED, YELLOW, GREEN;
}

// With fields, constructor, and methods
public enum Planet {
    EARTH(5.976e24, 6.37814e6),
    MARS(6.421e23, 3.3972e6);

    private final double mass;
    private final double radius;

    Planet(double mass, double radius) {
        this.mass = mass;
        this.radius = radius;
    }

    public double surfaceGravity() {
        return 6.67300E-11 * mass / (radius * radius);
    }
}

// Constant-specific class body
public enum Operation {
    PLUS {
        public int apply(int a, int b) { return a + b; }
    },
    MINUS {
        public int apply(int a, int b) { return a - b; }
    };

    public abstract int apply(int a, int b);
}

// Implementing an interface
public interface Describable { String describe(); }

public enum Role implements Describable {
    ADMIN, USER;
    public String describe() { return "Role: " + name(); }
}

// Enum singleton
public enum ConfigManager {
    INSTANCE;
    private final Map<String,String> settings = new HashMap<>();
    public String get(String key) { return settings.get(key); }
}
```

}

Member / API	Purpose
<code>values()</code>	Returns array of all constants, in declaration order
<code>valueOf(String)</code>	Returns the constant matching the given name, throws <code>IllegalArgumentException</code> otherwise
<code>name()</code>	Exact declared identifier of the constant (final, not overridable)
<code>ordinal()</code>	Zero-based declaration position
<code>compareTo(E)</code>	Compares by ordinal
<code>equals(Object)</code>	Identity comparison (same as <code>==</code> for enums)
<code>toString()</code>	Defaults to <code>name()</code> , safely overridable for display text
<code>switch</code>	Case labels use bare constant names, no qualification
<code>EnumSet</code>	Bit-vector backed <code>Set</code> implementation, only for enum types
<code>EnumMap</code>	Array backed <code>Map</code> implementation keyed by enum, ordered by ordinal
Serialization	Enums serialize by <code>name()</code> automatically; safe across JVMs by default

Key rules

- An enum constructor is implicitly private; enum constants cannot be instantiated externally.
- Enum types cannot extend another class (they implicitly extend `java.lang.Enum`) but can implement interfaces.
- `==` is always safe for enum comparison because each constant is a singleton per JVM/class-loader.
- A constant-specific class body creates an anonymous subclass – useful for enforcing per-constant behavior at compile time.

Python Enum Cheat Sheet

```
from enum import Enum, IntEnum, StrEnum, Flag, IntFlag, auto, unique

class Signal(Enum):
    RED = auto()
    YELLOW = auto()
    GREEN = auto()

class Priority(IntEnum):
    LOW = 1
    MEDIUM = 2
    HIGH = 3

class Suit(StrEnum):
    HEARTS = "hearts"
    SPADES = "spades"

class Permission(Flag):
    READ = auto()
    WRITE = auto()
    EXECUTE = auto()
    ALL = READ | WRITE | EXECUTE

class Planet(Enum):
    EARTH = (5.976e24, 6.37814e6)
    MARS = (6.421e23, 3.3972e6)

    def __init__(self, mass, radius):
        self.mass = mass
        self.radius = radius

    def surface_gravity(self):
        return 6.67300E-11 * self.mass / (self.radius ** 2)

    def __str__(self):
        return self.name.title()

@unique
class Status(Enum):
    ACTIVE = 1
    INACTIVE = 2

class Grade(Enum):
    A = 90
    B = 80

    @classmethod
    def _missing_(cls, value):
```



```

if isinstance(value, int) and value > 90:
    return cls.A
return None

```

Member / API	Purpose
<code>Enum</code>	Base class, members are unique objects, not comparable to <code>int</code> by default
<code>IntEnum</code>	Members behave as <code>int</code> , comparable and usable in arithmetic contexts
<code>StrEnum</code> (3.11+)	Members behave as <code>str</code>
<code>Flag</code> / <code>IntFlag</code>	Support bitwise <code> </code> , <code>&</code> , <code>^</code> combination of members
<code>auto()</code>	Assigns the next available value automatically
<code>.name</code> / <code>.value</code>	Identifier string / underlying stored value
<code>__init__</code>	Called once per member during class construction, attaches extra fields
<code>__str__</code>	Overridable for custom display; does not affect <code>repr</code>
<code>_missing_()</code>	Classmethod hook invoked when a lookup value has no direct match
<code>@unique</code>	Class decorator that raises <code>ValueError</code> on duplicate values
Aliases	Members sharing a value become aliases of the first-declared member

Key rules

- Members are singletons: `Signal.RED is Signal.RED` is always `True`.
- Plain `Enum` members are *not* orderable and are not equal to their raw value.
- Iterating a class iterates unique canonical members only; aliases are skipped.
- `match` statements can pattern-match directly on enum members as of Python 3.10.

Java vs Python Enum Comparison

Capability	Java	Python
Basic declaration	<code>enum X { A, B }</code>	<code>class X(Enum): A = auto()</code>
Underlying storage	Compiler-generated class instances	Any hashable value, often <code>int</code> or <code>str</code>
Int-like behavior	Not automatic; use <code>ordinal()</code> manually	<code>IntEnum</code>
String-like behavior	Override <code>toString()</code>	<code>StrEnum</code> or override <code>__str__</code>
Bit flags	Manual bitmasking, or <code>EnumSet</code>	<code>Flag</code> / <code>IntFlag</code> with native <code> </code>
Fields per constant	Constructor + instance fields	<code>__init__</code> + instance attributes
Per-constant behavior	Constant-specific class body (anonymous subclass)	Per-member logic inside shared methods, or <code>_missing_</code>
Interfaces / protocols	<code>implements</code>	Multiple inheritance with <code>Enum</code> mixins
Specialized collections	<code>EnumSet</code> , <code>EnumMap</code>	Regular <code>set</code> / <code>dict</code> ; members are already hashable
Duplicate values	Not directly representable (each constant is distinct)	Automatic aliasing, or <code>@unique</code> to forbid it
Unknown value handling	<code>valueOf</code> throws <code>IllegalArgumentException</code>	<code>_missing_</code> hook allows graceful fallback
Singleton pattern	Canonical singleton implementation target	Rare; module-level constants usually suffice
Serialization	By <code>name()</code> , JVM-safe	By <code>value</code> (JSON needs custom encoder)

This workbook revisits this table constantly. Every phase closes with a Java-vs-Python differences section that expands specific rows above.

Chapter 1

Phase 1 – Basic Enums

This phase establishes the mechanics of declaring an enum and using it as a safer replacement for magic constants, in both languages.

Q1: Traffic Signal

Difficulty: **Easy** **Phase:** 1 **Estimated Time:** 15 min
Concepts Covered: enum declaration, values(), basic switch

Problem Statement

A traffic light cycles through three states: red, yellow, and green. Model the light so that calling a “next state” operation always returns the correct following state, wrapping from green back to red. Using a plain integer or string to represent the state invites invalid values (e.g. 4 or “purple”); an enum makes invalid states unrepresentable.

Task

Define a `Signal` enum with constants `RED`, `YELLOW`, `GREEN`. Implement a function that, given the current signal, returns the next signal in the cycle.

Java Function Signature

```
public static Signal next(Signal current);
```

Python Function Signature

```
def next_signal(current: Signal) -> Signal: ...
```

Input

A single `Signal` value.

Output

The `Signal` value that follows it in the cycle `RED → GREEN → YELLOW → RED`.

Constraints

Only the three defined constants are valid input.

Examples

- Input: `RED` → Output: `GREEN` – red always transitions to green.
- Input: `GREEN` → Output: `YELLOW` – green always transitions to yellow.
- Input: `YELLOW` → Output: `RED` – yellow wraps back to red.

Requirements

- Must use an enum type, not integers or strings.
- Must use a `switch` (Java) / structural dispatch (Python) over the enum, not `ordinal()` arithmetic.

Enum Features Practiced

Java: enum declaration, constants, switch. **Python:** Enum, `auto()`.

Learning Objectives

Declare a minimal enum; understand that enum constants are the only valid values of the type.

Hints

Consider whether a `switch/if` chain or a lookup table best expresses a fixed cyclic order.

Follow-ups

1. Add a `previous()` operation.
2. Replace the switch with a data-driven approach once you reach Phase 3 (fields).

Interview Notes

Tests whether a candidate reaches for an enum instead of int constants when the domain is fixed and small. A common mistake is hardcoding transition order with `ordinal() + 1 mod 3`, which is fragile if constants are reordered.

Q2: Days of Week

Difficulty: **Easy** **Phase:** 1 **Estimated Time:** 15 min
Concepts Covered: enum declaration, `values()`, iteration

Problem Statement

You need to determine whether a given day is a weekday or a weekend, and to list all weekend days. Enumerating days as an enum lets you iterate over the fixed set safely.

Task

Define a `Day` enum for all seven days. Implement a function returning `true` if a day is a weekend, and a function returning the list of all weekend days by iterating `values()`.

Java Function Signature

```
public static boolean isWeekend(Day day);  
public static List<Day> weekendDays();
```

Python Function Signature

```
def is_weekend(day: Day) -> bool: ...  
def weekend_days() -> list[Day]: ...
```

Input

A `Day` value (for `isWeekend`); no input for `weekendDays`.

Output

A boolean, or a list of `Day` values.

Constraints

Seven distinct constants, declared Monday through Sunday.

Examples

- Input: `SATURDAY` → Output: `true`.
- Input: `WEDNESDAY` → Output: `false`.
- `weekendDays()` → Output: `[SATURDAY, SUNDAY]`, in declaration order.

Requirements

- Must iterate using `values()` (Java) or class iteration (Python) – no hardcoded list literal for `weekendDays`.

Enum Features Practiced

Java: `values()`, enhanced for-loop. **Python:** class iteration, membership testing.

Learning Objectives

Iterate over all constants of an enum type in declaration order.

Hints

`values()` always returns constants in the order they were declared, which you can rely on.

Follow-ups

1. Add a method that returns the day N positions ahead, wrapping around the week.
2. Group days by first letter using a `Map/dict`.

Interview Notes

Checks comfort with enum iteration versus hardcoded arrays. A common mistake is re-declaring the weekend list as a separate array that can drift out of sync with the enum.

Q3: Compass Direction

Difficulty: **Easy** **Phase:** 1 **Estimated Time:** 15 min

Concepts Covered: enum constants, `ordinal()`, modular arithmetic

Problem Statement

A robot faces one of four compass directions. Turning right or left should cycle through **NORTH**, **EAST**, **SOUTH**, **WEST** in order. `ordinal()` gives a natural numeric position for this kind of modular rotation.

Task

Define a `Direction` enum. Implement `turnRight` and `turnLeft` using ordinal-based modular arithmetic.

Java Function Signature

```
public static Direction turnRight(Direction current);  
public static Direction turnLeft(Direction current);
```

Python Function Signature

```
def turn_right(current: Direction) -> Direction: ...  
def turn_left(current: Direction) -> Direction: ...
```

Input

A `Direction` value.

Output

The resulting `Direction` after a 90-degree turn.

Constraints

Exactly four constants, declared clockwise starting at **NORTH**.

Examples

- Input: **NORTH**, `turnRight` → Output: **EAST**.
- Input: **WEST**, `turnRight` → Output: **NORTH** – wraps around.
- Input: **NORTH**, `turnLeft` → Output: **WEST** – wraps backward.

Requirements

- Must use `ordinal()` / `list(Direction)` indexing with modulo arithmetic, not a switch table.

Enum Features Practiced

Java: `ordinal()`, `values()` array indexing. **Python:** member list construction, index arithmetic.

Learning Objectives

Use ordinal position for cyclic navigation, and recognize its fragility if constants are reordered.

Hints

`values()[ordinal + 1 % values().length]` is the general pattern in Java.

Follow-ups

1. Implement `turn180()` in terms of two calls to `turnRight`.
2. Discuss why relying on `ordinal()` for business logic is discouraged in later phases.

Interview Notes

Interviewers use this to introduce the ordinal-fragility trap: reordering constants silently breaks ordinal-dependent logic. Best practice, covered later, is to store explicit direction vectors instead.

Q4: Chess Piece Color

Difficulty: **Easy** **Phase:** 1 **Estimated Time:** 10 min
Concepts Covered: enum constants, equals/==

Problem Statement

A chess piece is either `WHITE` or `BLACK`. Given a color, determine the opposing color – a two-constant enum is the simplest possible demonstration of type-safe binary state.

Task

Define a `Color` enum with `WHITE` and `BLACK`. Implement `opposite(Color)`.

Java Function Signature

```
public static Color opposite(Color color);
```

Python Function Signature

```
def opposite(color: Color) -> Color: ...
```

Input

A `Color` value.

Output

The other `Color` value.

Constraints

Exactly two constants.

Examples

- Input: `WHITE` → Output: `BLACK`.
- Input: `BLACK` → Output: `WHITE`.
- Calling `opposite(opposite(WHITE))` → Output: `WHITE` – involution property.

Requirements

- Must compare using `==` (Java) / `is` or `==` (Python), demonstrating enum identity comparison.

Enum Features Practiced

Java: `==` identity comparison. **Python:** identity via `is`, value equality via `==`.

Learning Objectives

Understand why `==` is always safe for Java enum comparison, and why Python enum members are also safe with `==` due to singleton identity.

Hints

No hint needed – this is a direct two-branch mapping.

Follow-ups

1. Extend to a **Player** concept that pairs a name with a **Color** in Phase 3.

Interview Notes

A classic warm-up. The interview point is confirming the candidate knows `==` is always safe for enums (unlike general Java objects, where `.equals()` is normally required).

Q5: Seasons

Difficulty: **Easy** **Phase:** 1 **Estimated Time:** 15 min

Concepts Covered: enum constants, `valueOf()`, exception handling

Problem Statement

Given a season name typed by a user as a string, convert it to the corresponding **Season** enum constant, or report that the input was invalid. This is the standard pattern for parsing external string input into a type-safe enum.

Task

Define a **Season** enum: **WINTER**, **SPRING**, **SUMMER**, **FALL**. Implement a parser that converts a string to a **Season**, handling invalid input gracefully.

Java Function Signature

```
public static Optional<Season> parseSeason(String input);
```

Python Function Signature

```
def parse_season(input: str) -> Season | None: ...
```

Input

A string, possibly not matching any constant name.

Output

The matching **Season**, or an empty/**None** result.

Constraints

Input matching must be exact-case for this exercise (case-insensitivity is covered in Q14).

Examples

- Input: "SUMMER" → Output: `Season.SUMMER`.
- Input: "AUTUMN" → Output: empty – not a declared constant name.
- Input: "summer" → Output: empty – case mismatch, by design in this exercise.

Requirements

- Must use `Enum.valueOf()` (Java) / `Season[input]` or `Season(input)` (Python) internally.
- Must not let an unhandled exception propagate for invalid input.

Enum Features Practiced

Java: `valueOf()`, `IllegalArgumentException` handling. **Python:** member access by name, `KeyError/ValueError` handling.

Learning Objectives

Safely convert external string data into an enum without crashing on bad input.

Hints

In Java, wrap `valueOf` in a try/catch. In Python, `Season[input]` looks up by name and raises `KeyError`, while `Season(input)` looks up by value.

Follow-ups

1. Add case-insensitive matching without using `valueOf` directly.
2. Return a descriptive error message instead of an empty result.

Interview Notes

Frequently asked because it surfaces a real production bug: naive use of `valueOf/Season[x]` on unsanitized input crashes the request thread. Best practice is always to guard the lookup.

Q6: Count Enum Members

Difficulty: **Easy** **Phase:** 1 **Estimated Time:** 10 min
Concepts Covered: `values()`, `len()`, reflection basics

Problem Statement

Generic tooling (for example, a UI dropdown generator) often needs to know how many constants an enum type declares, without hardcoding that number.

Task

Given any enum type, implement a function that returns its number of constants.

Java Function Signature

```
public static <E extends Enum<E>> int countConstants(Class<E> enumClass)
    ;
```

Python Function Signature

```
def count_constants(enum_cls: type[Enum]) -> int: ...
```

Input

An enum class/type reference.

Output

An integer count of declared constants.

Constraints

Must work for any enum type passed in, not just one hardcoded type.

Examples

- Input: `Signal.class` → Output: 3.
- Input: `Day.class` → Output: 7.
- Input: `Color.class` → Output: 2.

Requirements

- Must be generic over the enum type – no per-enum special casing.

Enum Features Practiced

Java: generic methods bounded by `Enum<E>`, `values().length`. **Python:** `len(list(enum_cls))`.

Learning Objectives

Write enum-generic utility code, a precursor to writing reusable enum-based libraries.

Hints

Java requires `Class<E>.getEnumConstants()` when you only have a `Class` object and no instance to call `values()` on directly.

Follow-ups

1. Write a generic function that returns the constant names as a `List<String>`.

Interview Notes

Tests whether the candidate knows `values()` is a compiler-synthesized static method that cannot be referenced through a generic type parameter directly, requiring `getEnumConstants()` instead. A common mistake is trying `E.values()`, which does not compile.

Phase 1 Summary

What was learned: declaring enums, iterating with `values()`, parsing with `valueOf()`, ordinal-based rotation, and identity comparison with `==`.

Common mistakes: using `ordinal()` for business-meaningful ordering that later needs constants inserted out of order; letting `valueOf/Season[x]` throw on unsanitized input; re-declaring constant lists that duplicate `values()`.

Java vs Python differences: Java's `values()` is a compiler-synthesized static method, unavailable through a generic type parameter without reflection; Python simply iterates the class. Java throws `IllegalArgumentException` on a bad `valueOf`; Python raises `KeyError` (by name) or `ValueError` (by value) depending on lookup style.

Interview tips: always mention that `==` is safe for Java enums, and that ordinal-based logic is fragile against reordering.

Mini review: before continuing, confirm you can declare an enum from scratch in both languages, iterate it, parse a string into it safely, and explain the ordinal fragility trap without notes.

Chapter 2

Phase 2 – Built-in Enum Methods

This phase drills the standard method surface every enum exposes: comparison, string conversion, and name-based lookup, including case-insensitive variants.

Q7: Enum Menu

Difficulty: **Easy** **Phase:** 2 **Estimated Time:** 15 min
Concepts Covered: `values()`, `toString()`, display formatting

Problem Statement

A CLI application must print a numbered menu of options derived from an enum, e.g. `MenuOption` constants `NEW_GAME`, `LOAD_GAME`, `SETTINGS`, `EXIT`, formatted as readable text rather than raw constant names.

Task

Implement a function that renders all constants as a numbered, human-readable menu string, one option per line, using 1-based numbering aligned to `values()` order.

Java Function Signature

```
public static String renderMenu();
```

Python Function Signature

```
def render_menu() -> str: ...
```

Input

None (operates over all constants).

Output

A multi-line string, one numbered option per line.

Constraints

Numbering must match declaration order exactly.

Examples

- Output line 1: "1. New Game" – underscore replaced with space, title case.
- Output line 3: "3. Settings".
- Four constants → four lines, no trailing blank line.

Requirements

- Must derive text from `name()`/`.name`, not a separate hardcoded string table.
- Must use `ordinal()` or enumeration index for numbering, not a manual counter.

Enum Features Practiced

Java: `values()`, `ordinal()`, `name()`. **Python:** class iteration, `.name`.

Learning Objectives

Combine iteration and naming APIs to generate UI text programmatically.

Hints

String transformation (underscore-to-space, title case) belongs outside the enum in this exercise – keep the enum itself simple until Phase 4 introduces enum-owned methods.

Follow-ups

1. Support selecting an option by typing its number back.
2. Move the display-name logic into the enum itself (preview of Phase 4).

Interview Notes

Checks whether a candidate needlessly hardcodes display strings that duplicate constant names, an easy source of drift.

Q8: Command Parser

Difficulty: Easy **Phase:** 2 **Estimated Time:** 20 min
Concepts Covered: `valueOf()`, Optional/None handling

Problem Statement

A text-based game reads player commands like "MOVE", "ATTACK", "DEFEND", "QUIT". Convert raw input into a `Command` enum for dispatch.

Task

Implement a parser converting a string to a `Command`, returning a well-defined absent value for unrecognized input (do not throw to the caller).

Java Function Signature

```
public static Optional<Command> parse(String raw);
```

Python Function Signature

```
def parse(raw: str) -> Command | None: ...
```

Input

Raw command string, exact case.

Output

Matching `Command` or an absent value.

Constraints

Four valid commands.

Examples

- Input: "ATTACK" → Output: `Command.ATTACK`.
- Input: "JUMP" → Output: absent – undefined command.
- Input: "" → Output: absent – empty string never matches.

Requirements

- Must not let `IllegalArgumentException/KeyError` propagate to the caller.

Enum Features Practiced

Java: `valueOf()`, exception-to-Optional conversion. **Python:** dict-style member lookup, exception handling.

Learning Objectives

Build a reusable safe-parse pattern for enum input, foundational for Q9.

Hints

A tiny private try/except wrapper around the raw lookup is the entire solution; resist adding a manual if/else chain over all constants.

Follow-ups

1. Reuse this parser inside a REPL loop that dispatches on the returned command.

Interview Notes

Nearly identical to Q5 by design – interviewers repeat this pattern to see if the candidate recognizes it and reuses the safe-parse idiom without re-deriving it.

Q9: Invalid Command

Difficulty: **Easy** **Phase:** 2 **Estimated Time:** 15 min
Concepts Covered: `valueOf()` exception details, error design

Problem Statement

Building on Q8, the game must show the player *why* a command was rejected and *what* the valid commands are, not just that it failed.

Task

Implement a function that, given invalid input, returns a formatted error message listing all valid command names.

Java Function Signature

```
public static String invalidCommandMessage(String raw);
```

Python Function Signature

```
def invalid_command_message(raw: str) -> str: ...
```

Input

An invalid raw command string.

Output

An error string naming the invalid input and listing valid options.

Constraints

Assume the caller already confirmed the input is invalid via Q8's parser.

Examples

- Input: "JUMP" → Output: "Unknown command 'JUMP'. Valid: MOVE, ATTACK, DEFEND, QUIT".
- Input: "atk" → Output: same style message referencing 'atk'.
- Valid list order must match declaration order.

Requirements

- Must build the valid-options list from `values()`, never a hardcoded literal.

Enum Features Practiced

Java: `values()`, `String.join`. **Python:** class iteration, `"`, `".join`.

Learning Objectives

Turn enum metadata into actionable, user-facing error messages.

Hints

Stream/list-comprehend `values()` directly into the joined string.

Follow-ups

1. Suggest the closest valid command using string-distance matching.

Interview Notes

Evaluates attention to UX in error handling, a frequently overlooked but commonly graded detail in take-home assignments.

Q10: Compare Priorities

Difficulty: Easy **Phase:** 2 **Estimated Time:** 15 min
Concepts Covered: compareTo(), Comparable, sorting

Problem Statement

A task queue holds tasks tagged LOW, MEDIUM, HIGH, CRITICAL. Tasks must be sortable by priority, highest first, using the enum's natural ordering.

Task

Implement a function that sorts a list of tasks (priority-tagged) from highest to lowest priority using enum comparison.

Java Function Signature

```
public static List<Task> sortByPriorityDesc(List<Task> tasks);
```

Python Function Signature

```
def sort_by_priority_desc(tasks: list[Task]) -> list[Task]: ...
```

Input

A list of tasks, each with a Priority field.

Output

The same tasks, sorted descending by priority.

Constraints

Priority declared in ascending order: LOW < MEDIUM < HIGH < CRITICAL.

Examples

- Input priorities [LOW, CRITICAL, MEDIUM] → Output order [CRITICAL, MEDIUM, LOW].
- All-equal priorities → original relative order preserved (stable sort).
- Single-element list → unchanged.

Requirements

- Must use compareTo() (Java) or IntEnum/rich comparison (Python), not manual ordinal subtraction exposed in application code.

Enum Features Practiced

Java: compareTo(), Comparator.reverseOrder(). **Python:** IntEnum ordering, sorted(..., reverse=True).

Learning Objectives

Use built-in ordinal-based comparison correctly and understand when plain Enum in Python needs to become IntEnum to be orderable.

Hints

Plain Python Enum members raise `TypeError` on `<`; check whether `Priority` needs to inherit from `IntEnum` for this exercise.

Follow-ups

1. Sort by priority, then by insertion timestamp as a tiebreaker.

Interview Notes

Common trap: candidates assume all Python enums are orderable like Java's. Interviewers probe whether the candidate knows plain Enum is not comparable.

Q11: Month Number

Difficulty: Easy **Phase:** 2 **Estimated Time:** 15 min
Concepts Covered: ordinal(), 1-based conversion

Problem Statement

Convert a Month enum constant to its conventional 1-based calendar number (January = 1, not 0), a direct but important off-by-one exercise using `ordinal()`.

Task

Define a Month enum (12 constants) and implement a conversion to 1-based month number, and its inverse.

Java Function Signature

```
public static int toNumber(Month month);  
public static Month fromNumber(int number);
```

Python Function Signature

```
def to_number(month: Month) -> int: ...  
def from_number(number: int) -> Month: ...
```

Input

A Month value, or an integer 1–12.

Output

The corresponding integer or Month.

Constraints

Valid numbers are 1 through 12 inclusive; out-of-range must raise/throw explicitly.

Examples

- Input: JANUARY → Output: 1.
- Input: DECEMBER → Output: 12.
- Input: 13 to `fromNumber` → throws/raises – invalid month number.

Requirements

- Must derive the number from `ordinal()` (+1), not a separately maintained field.

Enum Features Practiced

Java: `ordinal()`, `values()` bounds checking. **Python:** list indexing, explicit `ValueError`.

Learning Objectives

Practice safe off-by-one conversions between ordinal and domain-meaningful numbering.

Hints

Validate the input range before indexing to avoid an `ArrayIndexOutOfBoundsException/IndexError`.

Follow-ups

1. Add `daysInMonth()` (ignore leap years for now; revisit in Phase 3 with fields).

Interview Notes

Simple, but candidates frequently forget the +1/-1 offset or fail to validate bounds – interviewers watch for both.

Q12: Equality Test

Difficulty: **Easy** **Phase:** 2 **Estimated Time:** 10 min
Concepts Covered: `equals()`, `==`, `hashCode` consistency

Problem Statement

Demonstrate, and unit-test, that enum equality behaves consistently whether checked with `==` or `.equals()` in Java, and `==` or `is` in Python – an important guarantee when enums are used as `HashMap/dict` keys.

Task

Implement a function that returns `true` only if two references point to the exact same constant, verified two different ways, and asserts they always agree.

Java Function Signature

```
public static boolean sameConstant(Signal a, Signal b);
```

Python Function Signature

```
def same_constant(a: Signal, b: Signal) -> bool: ...
```

Input

Two `Signal` values.

Output

Boolean, true if both comparison strategies agree they are equal.

Constraints

Both comparisons (identity and equals-based) must be exercised inside the function.

Examples

- Input: `(RED, RED)` → Output: `true`.
- Input: `(RED, GREEN)` → Output: `false`.
- Reassigning a variable from a `values()` lookup still compares equal to the original constant.

Requirements

- Must assert (or return false if they ever disagree) that `==` and `.equals()/is` produce the same result for every pair.

Enum Features Practiced

Java: `==`, `.equals()`, singleton guarantee. **Python:** `is`, `==`, singleton guarantee.

Learning Objectives

Internalize why enum singletons make identity and equality checks interchangeable.

Hints

There is exactly one instance of each constant per classloader/process, by language design.

Follow-ups

1. Explain what would break this guarantee (e.g. deserialization into a new classloader in Java, or manual object construction bypassing the enum machinery).

Interview Notes

Interviewers use this to confirm the candidate would never write `.equals(null)` unguarded, and understands enum singleton guarantees underlie safe `==` usage.

Q13: Override `toString` / `__str__`

Difficulty: **Easy** **Phase:** 2 **Estimated Time:** 15 min
Concepts Covered: `toString()` override, `__str__`

Problem Statement

Raw constant names like `HIGH_PRIORITY` are not fit for end-user display. Override the default string conversion to produce a human-friendly label while keeping `name()` intact for internal logic.

Task

Override `toString()` / `__str__` on a `Priority` enum (`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`) to render as title case with spaces.

Java Function Signature

```
public enum Priority { LOW, MEDIUM, HIGH, CRITICAL;
    @Override public String toString() { /* implement */ }
}
```

Python Function Signature

```
class Priority(Enum):
    LOW, MEDIUM, HIGH, CRITICAL = ...
    def __str__(self) -> str: ...
```

Input

None; behavior is observed by printing/str-ing a constant.

Output

A human-readable label string.

Constraints

`name()` / `.name` must remain unchanged and still return `"CRITICAL"` etc.

Examples

- `Priority.CRITICAL.toString()` → `"Critical"`.
- `str(Priority.HIGH)` → `"High"`.
- `Priority.LOW.name()` → still `"LOW"` – unaffected by the override.

Requirements

- Must not modify `name()` / `.name`.
- Must override the display method, not add a separate differently-named method.

Enum Features Practiced

Java: `toString()` override. **Python:** `__str__` override.

Learning Objectives

Separate the machine-readable identifier (`name()`) from the human-readable presentation (`toString()` / `__str__`).

Hints

`name().charAt(0) + name().substring(1).toLowerCase()` handles single-word names; plan for multi-word names using underscores in the follow-up.

Follow-ups

1. Support multi-word constants like `OUT_OF_STOCK` → "Out Of Stock".
2. Discuss why `valueOf()` must still use `name()`, not the overridden string.

Interview Notes

Tests whether the candidate confuses `name()` (final, used for parsing) with `toString()` (overridable, used for display) – a very common real-world bug source.

Q14: Case-Insensitive Parser

Difficulty: Easy **Phase:** 2 **Estimated Time:** 20 min

Concepts Covered: `valueOf()` limitations, custom lookup, Map caching

Problem Statement

Real user input arrives inconsistently cased ("summer", "Summer", "SUMMER"). `valueOf()/name-`lookup is case-sensitive, so a custom, efficient lookup is required.

Task

Implement a case-insensitive parser for the `Season` enum from Q5 that works regardless of input casing, without scanning `values()` on every call.

Java Function Signature

```
public static Optional<Season> parseIgnoreCase(String input);
```

Python Function Signature

```
def parse_ignore_case(input: str) -> Season | None: ...
```

Input

A string in arbitrary case.

Output

Matching `Season` or an absent value.

Constraints

Lookup should be $O(1)$ after any one-time setup cost, not $O(n)$ per call.

Examples

- Input: "summer" → Output: `Season.SUMMER`.
- Input: "WiNtEr" → Output: `Season.WINTER`.
- Input: "autumn" → Output: absent – not a declared name.

Requirements

- Must precompute a lookup structure once (e.g. a static/module-level map keyed by lowercase name), not re-scan `values()` inside the function body on every call.

Enum Features Practiced

Java: static initializer blocks, `Map<String,Season>`. **Python:** module-level dict comprehension over the class.

Learning Objectives

Build a cached, case-insensitive enum lookup – a pattern reused constantly in Phase 8.

Hints

Build the map once, keyed by `name().toLowerCase()`, outside the hot path.

Follow-ups

1. Extend the cache pattern to also match on `toString()` display values.
2. Benchmark this against a naive per-call scan (conceptually, no code needed).

Interview Notes

A very common real interview question. The key insight interviewers want is precomputing the lookup map rather than looping over `values()` per call.

Phase 2 Summary

What was learned: safe string-to-enum parsing, natural ordering via `compareTo()/IntEnum`, overriding display text without breaking `name()`-based parsing, and building cached case-insensitive lookups.

Common mistakes: conflating `toString()` with `name()`; assuming plain Python `Enum` supports ordering; re-scanning `values()` on every parse call instead of caching a lookup map.

Java vs Python differences: Java enums are `Comparable` by default via ordinal; Python's base `Enum` is not orderable and requires `IntEnum` (or a custom `__lt__`) for comparison operators.

Interview tips: always separate parsing (`name()`-based, must stay stable) from display (`toString()`, safe to change) in your explanation.

Mini review: confirm you can implement a cached case-insensitive parser from memory and explain, without hesitation, the difference between `name()` and `toString()`.

Chapter 3

Phase 3 – Fields & Constructors

This phase introduces enum constructors, instance fields, and the pattern of enriching each constant with associated data.

Q15: Planet Gravity

Difficulty: **Easy** **Phase:** 3 **Estimated Time:** 20 min
Concepts Covered: enum constructors, instance fields, methods

Problem Statement

Each planet has a mass and radius. Surface gravity calculation requires both values. Rather than storing them in a separate lookup table, attach them directly to each constant via a constructor and fields.

Task

Define a `Planet` enum with constants for Earth, Mars, Venus, and Jupiter, each storing mass and radius. Implement a `surfaceGravity()` method.

Java Function Signature

```
public enum Planet {
    EARTH(5.976e24, 6.37814e6), MARS(6.421e23, 3.3972e6),
    VENUS(4.869e24, 6.05118e6), JUPITER(1.898e27, 7.1492e7);

    private final double mass, radius;
    Planet(double mass, double radius) { /* implement */ }
    public double surfaceGravity() { /* implement */ }
}
```

Python Function Signature

```
class Planet(Enum):
    EARTH = ...
    MARS = ...
    # etc

    def __init__(self, mass, radius):
        self.mass = mass
```

```
self.radius = radius

def surface_gravity(self) -> float: ...
```

Input

A `Planet` value and implicitly its attached data.

Output

The computed surface gravity ($G * \text{mass} / \text{radius}^2$).

Constraints

Must attach exactly two numeric fields per constant; no defaults or nulls.

Examples

- `Planet.EARTH.surfaceGravity()` → approximately 9.81.
- `Planet.JUPITER.surfaceGravity()` → approximately 24.79.
- Fields must be private final.

Requirements

- Constructor parameters must be explicitly listed in the constant declaration.
- Instance methods must access fields directly (`this.mass`), not a static lookup table.

Enum Features Practiced

Java: enum constructor, instance fields (final), instance methods. **Python:** `__init__`, instance attributes.

Learning Objectives

Understand that enum constants are class instances with their own fields and methods.

Hints

The gravitational constant is 6.67300E-11; verify your formula produces expected results for Earth ($\approx 9.81 \text{ m/s}^2$).

Follow-ups

1. Add a method to compute escape velocity.
2. Add a static method finding the planet with the highest gravity.

Interview Notes

Core question: does the candidate understand that enum instances can have fields and methods, not just be named integers? Watch for confusion with static fields.

Q16: HTTP Status

Difficulty: Medium **Phase:** 3 **Estimated Time:** 20 min
Concepts Covered: enum fields, toString override, factory method

Problem Statement

HTTP status codes (e.g. 200, 404, 500) pair a numeric code with a human-friendly message. Attach both to each enum constant and provide safe retrieval by code.

Task

Define a `Status` enum for common HTTP codes (200, 400, 401, 403, 404, 500, 503), each storing the code and description. Implement a method to find the status by code, and override `toString()` for display.

Java Function Signature

```
public enum Status {
    OK(200, "OK"), NOT_FOUND(404, "Not Found"),
    // ... other constants

    private final int code;
    private final String description;

    Status(int code, String description) { /* implement */ }
    public static Optional<Status> byCode(int code) { /* implement */ }
    @Override public String toString() { /* implement */ }
}
```

Python Function Signature

```
class Status(Enum):
    OK = (200, "OK")
    NOT_FOUND = (404, "Not Found")
    # ... etc

    def __init__(self, code: int, description: str): ...

    @classmethod
    def by_code(cls, code: int) -> Status | None: ...

    def __str__(self) -> str: ...
```

Input

A `Status` value, or an integer code.

Output

`Status` instance, or a formatted string representation for `toString()`.

Constraints

Seven constants; code lookup must handle invalid codes gracefully.

Examples

- `Status.NOT_FOUND.code` → 404.
- `Status.byCode(404)` → `Status.NOT_FOUND`.
- `Status.byCode(999)` → absent – no such status.
- `String.valueOf(Status.OK)` → "200 OK".

Requirements

- Must use a constructor storing code and description.
- `byCode` must not throw on invalid input; use `Optional/None`.
- `toString()` output must be “code description”, not the constant name.

Enum Features Practiced

Java: enum constructor, instance fields, static factory method, `Optional`. **Python:** `__init__`, `@classmethod`, `None`.

Learning Objectives

Combine multiple fields, static factory methods, and display customization into a single enum type.

Hints

The static factory `byCode` should iterate `values()` building a lookup, or cache a `Map` in a static initializer.

Follow-ups

1. Add a method returning a default status for a given HTTP error family (4xx $\rightarrow$ 400, 5xx $\rightarrow$ 500).
2. Discuss why we store code/description rather than deriving them from `ordinal()` or `name()`.

Interview Notes

Tests whether the candidate naturally reaches for a lookup map for reverse lookups, and understands that enum fields are per-constant.

Q17: Currency

Difficulty: **Medium** **Phase:** 3 **Estimated Time:** 25 min
Concepts Covered: enum fields, string parsing, exchange rates

Problem Statement

A currency has a code (e.g. “USD”), name, and symbol. Implement an enum supporting lookups by code and formatting amounts.

Task

Define a `Currency` enum for USD, EUR, GBP, JPY, storing code, name, and symbol. Provide methods to look up by code and format an amount as a string.

Java Function Signature

```
public enum Currency {
    USD("USD", "US Dollar", "$"),
    EUR("EUR", "Euro", "€"),
    // ... etc

    private final String code, name, symbol;
    Currency(String code, String name, String symbol) { /* implement */
    }
```

```
public static Optional<Currency> byCode(String code) { /* implement
    */ }
public String format(double amount) { /* implement */ }
}
```

Python Function Signature

```
class Currency(Enum):
    USD = ("USD", "US Dollar", "$")
    EUR = ("EUR", "Euro", "€")
    # ... etc

    def __init__(self, code: str, name: str, symbol: str): ...
    @classmethod
    def by_code(cls, code: str) -> Currency | None: ...
    def format(self, amount: float) -> str: ...
```

Input

A `Currency` value, a code string, or an amount to format.

Output

Matching `Currency`, or a formatted string like `$1,234.56`.

Constraints

Four constants; code lookup must be case-insensitive; formatting must include thousands separators and two decimal places.

Examples

- `Currency.byCode("usd") → Currency.USD`.
- `Currency.EUR.format(1234.5) → "€1,234.50"`.
- `Currency.JPY.format(10000) → "¥10,000"` (no decimal places).

Requirements

- Constructor must accept and store three strings.
- `byCode` lookup must be case-insensitive and cached.
- `format` output must handle symbol placement appropriately for the currency.

Enum Features Practiced

Java: constructor, cached lookup, `String.format`. **Python:** `__init__`, cached lookup, format strings.

Learning Objectives

Reuse the Q14 cached lookup pattern with multi-field enums.

Hints

Build the code-to-currency map once in a static/class-level context; precompute in lowercase for case-insensitive lookups.

Follow-ups

1. Add a field for the number of decimal places (USD/EUR: 2, JPY: 0) and use it in format.

Interview Notes

Extends Phase 2's cached lookup into multi-field enum territory, checking whether patterns generalize.

Q18: Coin Calculator

Difficulty: **Medium** **Phase:** 3 **Estimated Time:** 25 min
Concepts Covered: enum fields, arithmetic, filtering

Problem Statement

U.S. coins have a denomination and weight. Given a list of coins and a target monetary value, determine the minimum number of coins needed (a classic greedy algorithm problem, enhanced with enum-based coin representation).

Task

Define a `Coin` enum (`PENNY`, `NICKEL`, `DIME`, `QUARTER`) with cent value and weight. Implement a function calculating the minimum coin count to reach a target cents value.

Java Function Signature

```
public enum Coin {
    PENNY(1, 2.5), NICKEL(5, 5.0),
    DIME(10, 2.268), QUARTER(25, 5.67);

    private final int cents;
    private final double grams;

    Coin(int cents, double grams) { /* implement */ }
    public int getCents() { /* implement */ }
}

public static int minCoinsForAmount(int cents) { /* implement */ }
```

Python Function Signature

```
class Coin(Enum):
    PENNY = (1, 2.5)
    NICKEL = (5, 5.0)
    DIME = (10, 2.268)
    QUARTER = (25, 5.67)

    def __init__(self, cents: int, grams: float): ...

def min_coins_for_amount(cents: int) -> int: ...
```

Input

An integer number of cents.

Output

Minimum coin count, using standard greedy algorithm (largest denominations first).

Constraints

Input is always non-negative and representable; no fractional cents.

Examples

- Input: 41 cents → Output: 4 (one quarter, one dime, one nickel, one penny).
- Input: 25 cents → Output: 1 (one quarter).
- Input: 0 cents → Output: 0.

Requirements

- Must use the `Coin` enum constants in descending-value order.
- Must not hardcode coin values outside the enum.

Enum Features Practiced

Java: enum fields, accessor methods, iteration with `values()`. **Python:** enum attributes, class iteration.

Learning Objectives

Apply enums to classic algorithms, isolating data (coin definitions) from logic (greedy algorithm).

Hints

Iterate `values()` in reverse order (largest denomination first) for the greedy strategy.

Follow-ups

1. Refactor to handle arbitrary coin systems (e.g. euro coins).
2. Implement dynamic programming solution for minimum coins that works for any set of denominations.

Interview Notes

Tests whether the candidate separates enum definitions from algorithm logic, and whether they default to greedy (which is optimal for U.S. coinage but not all coin systems).

Q19: Employee Roles

Difficulty: **Medium** **Phase:** 3 **Estimated Time:** 25 min
Concepts Covered: enum fields, salary logic, business rules

Problem Statement

Each employee role (Engineer, Manager, HR, Finance) has a base salary, a bonus multiplier, and a department. Computing gross compensation for an employee requires data from both the role and the individual's tenure.

Task

Define a `Role` enum storing base salary, bonus rate, and department. Implement a method computing annual compensation given base salary and years of service (with a bonus accrual rule: 1% per year, capped at 10%).

Java Function Signature

```
public enum Role {
    ENGINEER(80000, 0.15, "Engineering"),
    MANAGER(100000, 0.25, "Management"),
    HR(60000, 0.10, "Human Resources"),
    FINANCE(85000, 0.20, "Finance");

    private final int baseSalary;
    private final double bonusRate;
    private final String department;

    Role(int baseSalary, double bonusRate, String dept) { /* implement
        */ }
    public double annualCompensation(int yearsOfService) { /* implement
        */ }
}
```

Python Function Signature

```
class Role(Enum):
    ENGINEER = (80000, 0.15, "Engineering")
    MANAGER = (100000, 0.25, "Management")
    HR = (60000, 0.10, "Human Resources")
    FINANCE = (85000, 0.20, "Finance")

    def __init__(self, base_salary: int, bonus_rate: float, dept: str):
        ...

    def annual_compensation(self, years_of_service: int) -> float: ...
```

Input

A `Role` value and years of service.

Output

Annual compensation, calculated as $\text{base} + (\text{base} \times \min(\text{yearsOfService} / 100, 0.10) \times \text{bonusRate})$.

Constraints

Years of service is non-negative; bonus accrual caps at 10%.

Examples

- `Role.ENGINEER.annualCompensation(0)` $\rightarrow$ 80000.
- `Role.ENGINEER.annualCompensation(10)` $\rightarrow$ approximately 81200 (1% bonus rate $\times$ 0.15 $\times$ 80000).
- `Role.MANAGER.annualCompensation(100)` $\rightarrow$ base + (10% $\times$ 0.25 $\times$ 100000).

Requirements

- Bonus rate must be applied per-role, not hardcoded.
- Accrual cap at 10% must be enforced.

Enum Features Practiced

Java: enum fields, instance methods with logic. **Python:** enum attributes, instance methods with arithmetic.

Learning Objectives

Apply enum fields to real business logic and confirm that enums are not just data carriers but full objects capable of hosting complex behavior.

Hints

Use `Math.min(yearsOfService % 100, 10)` to cap the bonus accrual percent at 10.

Follow-ups

1. Add a location field and implement location-based cost-of-living adjustments.
2. Extend to compute taxes or net pay.

Interview Notes

A realistic, domain-heavy problem checking whether the candidate naturally uses enums for business entity variants, not just collections of named constants.

Q20: File Types

Difficulty: Easy **Phase:** 3 **Estimated Time:** 20 min

Concepts Covered: enum fields, file extension mapping, lookups

Problem Statement

File types (Image, Document, Audio, Video, Spreadsheet) have extensions and MIME types. Map a file to its type, or query the list of extensions for a type.

Task

Define a `FileType` enum with fields for MIME type and common extensions. Provide methods to look up a file type by extension, and to list extensions for a type.

Java Function Signature

```
public enum FileType {
    IMAGE("image/jpeg", new String[]{"jpg", "png", "gif"}),
    DOCUMENT("application/pdf", new String[]{"pdf", "docx", "txt"}),
    // ... etc

    private final String mimeType;
    private final String[] extensions;

    FileType(String mimeType, String[] extensions) { /* implement */ }
    public static Optional<FileType> byExtension(String ext) { /*
        implement */ }
}
```

```
    public String[] getExtensions() { /* implement */ }  
}
```

Python Function Signature

```
class FileType(Enum):  
    IMAGE = ("image/jpeg", ["jpg", "png", "gif"])  
    DOCUMENT = ("application/pdf", ["pdf", "docx", "txt"])  
    # ... etc  
  
    def __init__(self, mime_type: str, extensions: list[str]): ...  
  
    @classmethod  
    def by_extension(cls, ext: str) -> FileType | None: ...  
  
    def get_extensions(self) -> list[str]: ...
```

Input

A file extension (with or without leading dot).

Output

Matching `FileType`, or the list of extensions for a type.

Constraints

Five types; extension lookup must be case-insensitive and handle leading dot gracefully.

Examples

- `FileType.byExtension("pdf")` → `FileType.DOCUMENT`.
- `FileType.byExtension(".PNG")` → `FileType.IMAGE` (normalized).
- `FileType.AUDIO.getExtensions()` → `["mp3", "wav", "flac"]`.

Requirements

- Must normalize input (lowercase, strip leading dot).
- Lookup must be cached; iterate only once.

Enum Features Practiced

Java: enum array fields, static factory, normalization. **Python:** enum list attributes, class-method factory.

Learning Objectives

Handle collections within enum fields, and build multi-to-one reverse lookups.

Hints

Build a `Map<String, FileType>` during static initialization, keyed by lowercase extension (without dot).

Follow-ups

1. Add a method to validate a filename against its declared type.

Interview Notes

Combines fields, collections, and reverse lookup pattern; commonly asked in medium-level interviews.

Q21: Countries

Difficulty: Easy **Phase:** 3 **Estimated Time:** 20 min
Concepts Covered: enum fields, locales, currency associations

Problem Statement

A country has a name, ISO code, and associated currency. Model the relationship directly in the enum, and provide lookups by code.

Task

Define a `Country` enum for USA, UK, France, Japan, Germany, storing ISO 3166-1 alpha-2 code and the `Currency` enum (from Q17) for each. Provide a lookup by ISO code.

Java Function Signature

```
public enum Country {
    USA("US", Currency.USD),
    UK("GB", Currency.GBP),
    FRANCE("FR", Currency.EUR),
    // ... etc

    private final String isoCode;
    private final Currency currency;

    Country(String isoCode, Currency currency) { /* implement */ }
    public static Optional<Country> byCode(String isoCode) { /*
        implement */ }
    public Currency getCurrency() { /* implement */ }
}
```

Python Function Signature

```
class Country(Enum):
    USA = ("US", Currency.USD)
    UK = ("GB", Currency.GBP)
    FRANCE = ("FR", Currency.EUR)
    # ... etc

    def __init__(self, iso_code: str, currency: Currency): ...
    @classmethod
    def by_code(cls, iso_code: str) -> Country | None: ...
    def get_currency(self) -> Currency: ...
```

Input

An ISO code string, or a `Country` value.

Output

Matching `Country`, or the associated `Currency`.

Constraints

Five countries; lookup must be case-insensitive.

Examples

- `Country.byCode("us") → Country.USA.`
- `Country.FRANCE.getCurrency() → Currency.EUR.`
- `Country.byCode("xx") → absent.`

Requirements

- Must store an enum-typed field (the `Currency`), demonstrating enum-to-enum references.
- Lookup must be cached.

Enum Features Practiced

Java: enum fields holding other enum instances. **Python:** enum attributes holding other enum members.

Learning Objectives

Compose multiple enums together in a type-safe manner.

Hints

Cache the code-to-country map in a static initializer.

Follow-ups

1. Query all countries using a given currency.

Interview Notes

Tests whether the candidate sees enums as composable types, not siloed.

Q22: Maze Directions (dx, dy)

Difficulty: Easy **Phase:** 3 **Estimated Time:** 20 min
Concepts Covered: enum fields, coordinate vectors, direction logic

Problem Statement

A maze solver needs to move North, South, East, West. Each direction maps to a change in (x, y) coordinates. Attach coordinate vectors to each direction enum constant for direct use in pathfinding.

Task

Define a `Direction` enum with fields for dx (column offset) and dy (row offset). Implement a method to apply a direction to a position.

Java Function Signature

```
public enum Direction {
    NORTH(0, -1), SOUTH(0, 1),
    EAST(1, 0), WEST(-1, 0);

    public final int dx, dy;
    Direction(int dx, int dy) { /* implement */ }
    public Position move(Position pos) { /* implement */ }
}

public record Position(int x, int y) {}
```

Python Function Signature

```
class Direction(Enum):
    NORTH = (0, -1)
    SOUTH = (0, 1)
    EAST = (1, 0)
    WEST = (-1, 0)

    def __init__(self, dx: int, dy: int): ...
    def move(self, pos: tuple[int, int]) -> tuple[int, int]: ...
```

Input

A Position/(x, y) tuple and a Direction.

Output

New position after applying the direction offset.

Constraints

Four constants; offsets are integers.

Examples

- Position (5, 5), direction NORTH $\rightarrow$ (5, 4).
- Position (5, 5), direction EAST $\rightarrow$ (6, 5).
- Applying NORTH twice from (5, 5) $\rightarrow$ (5, 3).

Requirements

- Must encode dx/dy in the enum, not compute or look up elsewhere.

Enum Features Practiced

Java: public final enum fields. **Python:** enum attributes accessible from members.

Learning Objectives

Attach derived data (coordinate deltas) to enum constants for use in algorithms.

Hints

Standard orientation: NORTH is (0, -1) if treating the top-left as (0, 0) with y increasing downward.

Follow-ups

1. Implement a method returning the opposite direction.
2. Implement a method rotating 90 degrees clockwise.

Interview Notes

A classic pattern in game/maze algorithm problems; interviewers watch whether the candidate naturally attaches data to enums or reaches for hardcoded arrays.

Phase 3 Summary

What was learned: enum constructors, instance fields (including collections and other enum types), instance methods accessing fields, and reverse lookups via static factories.

Common mistakes: confusing instance fields with static fields; forgetting to cache reverse lookups; hardcoding data outside the enum that should be part of the constant declarations.

Java vs Python differences: Java enum constructors are implicitly private and run once per constant during class initialization; Python uses `__init__` similarly. Java requires explicit field declarations and constructors; Python uses tuple unpacking in the value.

Interview tips: always explain why you attach data to the enum (single source of truth, tight coupling of constant and its properties) rather than maintaining a separate lookup table.

Mini review: confirm you can write a multi-field enum with a constructor and reverse lookup method without notes, and explain why this pattern beats a separate `HashMap`.

Chapter 4

Phase 4 – Enum Methods

This phase focuses on instance and static methods within enums, demonstrating that enums are full classes capable of hosting arbitrary logic.

Q23: Calculator

Difficulty: Medium **Phase:** 4 **Estimated Time:** 25 min
Concepts Covered: abstract enum methods, constant-specific behavior

Problem Statement

Implement a calculator over four operations (PLUS, MINUS, TIMES, DIVIDE). Each operation is represented as an enum constant, and each must know how to apply itself to two operands.

Task

Define an `Operation` enum with four constants. Define an abstract method `apply(int, int) -> int`. Each constant implements it via an anonymous subclass (constant-specific class body).

Java Function Signature

```
public enum Operation {  
    PLUS { int apply(int a, int b) { return a + b; } },  
    MINUS { int apply(int a, int b) { return a - b; } },  
    // ... etc  
  
    abstract int apply(int a, int b);  
}
```

Python Function Signature

```
class Operation(Enum):  
    PLUS = "+"  
    MINUS = "-"  
    TIMES = "*"  
    DIVIDE = "/"  
  
    def apply(self, a: int, b: int) -> int: ...
```

Input

An `Operation` and two integers.

Output

The result of the operation.

Constraints

`DIVIDE` must raise an exception on division by zero; other ops may not return fractional values in this exercise.

Examples

- `Operation.PLUS.apply(2, 3) → 5`.
- `Operation.TIMES.apply(4, 5) → 20`.
- `Operation.DIVIDE.apply(10, 0) → throws/raises`.

Requirements

- Java: must use constant-specific class body (each constant overrides the abstract method).
- Python: dispatch within a shared method (or use polymorphic methods per operation).

Enum Features Practiced

Java: abstract enum methods, constant-specific class body. **Python:** per-operation method logic dispatch.

Learning Objectives

Understand that Java enum constants can override abstract methods individually, enabling per-constant behavior at compile time.

Hints

Each `PLUS { }` block creates an anonymous subclass of `Operation` implementing the abstract method.

Follow-ups

1. Add a `toString()` that returns the operator symbol (`+`, `,`, `×`, `÷`).
2. Build a simple expression parser dispatching to `apply`.

Interview Notes

Core Java enum feature; if the candidate struggles here, they may not understand Java enums as classes. Python candidates will dispatch within a method, a valid alternative.

Q24: Roman Numerals

Difficulty: **Medium** **Phase:** 4 **Estimated Time:** 30 min
Concepts Covered: enum methods, string conversion, algorithm

Problem Statement

Convert between integers and Roman numerals. Each symbol (I, V, X, L, C, D, M) represents a value and has context-dependent rules (e.g. I before V means subtract, not add).

Task

Define a `RomanSymbol` enum with each symbol's integer value. Implement functions to convert an integer to Roman numerals and vice versa.

Java Function Signature

```
public enum RomanSymbol {
    I(1), V(5), X(10), L(50), C(100), D(500), M(1000);

    private final int value;
    RomanSymbol(int value) { this.value = value; }
    public int getValue() { return value; }
}

public static String toRoman(int num) { /* implement */ }
public static int fromRoman(String roman) { /* implement */ }
```

Python Function Signature

```
class RomanSymbol(IntEnum):
    I = 1
    V = 5
    X = 10
    L = 50
    C = 100
    D = 500
    M = 1000

def to_roman(num: int) -> str: ...
def from_roman(roman: str) -> int: ...
```

Input

An integer 1–3999, or a Roman numeral string.

Output

Roman numeral string, or the integer value.

Constraints

Input integers are always 1–3999; Roman strings are well-formed.

Examples

- `toRoman(1994)` → "MCMXCIV".
- `fromRoman("MCMXCIV")` → 1994.
- `toRoman(27)` → "XXVII".

Requirements

- Must use the `RomanSymbol` enum to drive conversions, not hardcoded strings.
- Must handle subtractive notation (IV = 4, IX = 9, XL = 40, XC = 90, CD = 400, CM = 900).

Enum Features Practiced

Java: enum fields, methods, greedy iteration. **Python:** IntEnum, value-based logic.

Learning Objectives

Use enum values as data driving a more complex algorithm, separate from the algorithm logic itself.

Hints

Build a sorted list of (value, symbol) pairs in descending order, including subtractive pairs; greedily peel off the largest value that fits into the remaining number.

Follow-ups

1. Extend to numbers beyond 3999 using vinculum notation (bar over a symbol).

Interview Notes

Assesses whether the candidate can isolate enum data from algorithm logic, and whether they'd hardcode magic strings in production.

Q25: Direction Rotation

Difficulty: Easy **Phase:** 4 **Estimated Time:** 15 min
Concepts Covered: enum methods, circular permutation

Problem Statement

Building on Q22 (Maze Directions), implement methods to rotate a direction 90, 180, 270 degrees clockwise, without hardcoding ordinal arithmetic.

Task

Add instance methods `rotateClockwise(int turns)` to the `Direction` enum, returning the direction after N 90-degree clockwise rotations.

Java Function Signature

```
public enum Direction {
    NORTH(0, -1), SOUTH(0, 1),
    EAST(1, 0), WEST(-1, 0);

    // ... fields and constructor from Q22

    public Direction rotateClockwise(int turns) { /* implement */ }
}
```

Python Function Signature

```
class Direction(Enum):
    NORTH = (0, -1)
    SOUTH = (0, 1)
    EAST = (1, 0)
    WEST = (-1, 0)
```



```
def __init__(self, dx: int, dy: int): ...  
def rotate_clockwise(self, turns: int) -> Direction: ...
```

Input

A `Direction` and number of 90-degree clockwise turns.

Output

The rotated direction.

Constraints

Turns can be negative (counterclockwise) or exceed 4.

Examples

- `Direction.NORTH.rotateClockwise(1)` → `EAST`.
- `Direction.NORTH.rotateClockwise(4)` → `NORTH` (full circle).
- `Direction.NORTH.rotateClockwise(-1)` → `WEST`.

Requirements

- Must not hardcode a lookup table outside the enum; use ordinal or list indexing with modulo.

Enum Features Practiced

Java: instance method accessing ordinal, `values()` lookup. **Python:** instance method using member list indexing.

Learning Objectives

Build circular navigation methods that are less fragile than raw ordinal arithmetic.

Hints

Use `(ordinal() + turns) % values().length` to compute the new ordinal; handle negative modulo carefully in Python.

Follow-ups

1. Add a static method `between(Direction a, Direction b)` returning the number of clockwise turns from `a` to `b`.

Interview Notes

Checks whether the candidate writes ordinal-safe instance methods, a useful skill for enums with circular structure.

Q26: File Extension Checker

Difficulty: Easy **Phase:** 4 **Estimated Time:** 15 min
Concepts Covered: enum methods, predicate methods, filtering

Problem Statement

Given a filename, determine its file type and whether it matches a set of allowed types. Attach a predicate method to the enum for this check.

Task

Add a method `matches(String)` to the `FileType` enum (from Q20) that returns true if a given filename has an extension matching one of the type's declared extensions.

Java Function Signature

```
public enum FileType {  
    // ... constants and fields from Q20  
  
    public boolean matches(String filename) { /* implement */ }  
}
```

Python Function Signature

```
class FileType(Enum):  
    # ... from Q20  
  
    def matches(self, filename: str) -> bool: ...
```

Input

A filename string and a `FileType`.

Output

Boolean, true if the filename's extension is in the type's extension list.

Constraints

Matching must be case-insensitive; filename may or may not have a leading dot in its extension.

Examples

- `FileType.IMAGE.matches("photo.JPG") → true.`
- `FileType.IMAGE.matches("data.txt") → false.`
- `FileType.DOCUMENT.matches("report.PDF") → true.`

Requirements

- Must use the enum's own extension list, not duplicate it in the method.

Enum Features Practiced

Java: instance method querying instance fields. **Python:** instance method accessing enum attributes.

Learning Objectives

Add behavior to enums that work with their own data.

Hints

Extract the extension from the filename, normalize to lowercase, then check membership in the enum's extension array/list.

Follow-ups

1. Add a static method that filters a list of filenames by type.

Interview Notes

Straightforward but realistic; interviewers note whether the candidate encapsulates logic in the enum (good) or leaves it to the caller (common oversight).

Q27: Temperature Converter

Difficulty: Easy **Phase:** 4 **Estimated Time:** 15 min

Concepts Covered: enum methods, static methods

Problem Statement

Convert temperature between Celsius and Fahrenheit. Attach methods to a `TempUnit` enum for conversion.

Task

Define a `TempUnit` enum (`CELSIUS`, `FAHRENHEIT`, `KELVIN`) and implement methods to convert a value from one unit to another.

Java Function Signature

```
public enum TempUnit {
    CELSIUS, FAHRENHEIT, KELVIN;

    public double convert(double value, TempUnit to) { /* implement */ }
}
```

Python Function Signature

```
class TempUnit(Enum):
    CELSIUS = "C"
    FAHRENHEIT = "F"
    KELVIN = "K"

    def convert(self, value: float, to: TempUnit) -> float: ...
```

Input

A numeric value, a source `TempUnit`, and a target `TempUnit`.

Output

The converted value.

Constraints

Absolute zero is 0 K, 273.15 °C, 459.67 °F; conversions must respect this.

Examples

- `CELSIUS.convert(0, FAHRENHEIT) → 32.0`.
- `FAHRENHEIT.convert(32, CELSIUS) → 0.0`.
- `CELSIUS.convert(0, KELVIN) → 273.15`.

Requirements

- Must handle all 9 conversion pairs (3×3 , including same-unit conversions).
- Must not hardcode the conversion formulas outside the enum.

Enum Features Practiced

Java: switch over enum, method dispatch. **Python:** method dispatch over enum member.

Learning Objectives

Use enums to drive multi-case logic safely.

Hints

Convert the input to Celsius first (an intermediate), then from Celsius to the target unit. This centralizes formulas.

Follow-ups

1. Refactor to use a helper enum or numeric constants for conversion factors.

Interview Notes

Checks whether the candidate centralizes conversion logic (good) or scatters it across the codebase (common mistake).

Q28: Discount Calculator

Difficulty: Easy **Phase:** 4 **Estimated Time:** 15 min
Concepts Covered: enum methods, field-driven logic

Problem Statement

An e-commerce site offers tiered customer discounts: BRONZE (5%), SILVER (10%), GOLD (15%), PLATINUM (25%). Attach a method to the enum calculating the discounted price.

Task

Define a `Tier` enum with a discount percentage field. Implement a method calculating the final price after applying the tier's discount.

Java Function Signature

```
public enum Tier {  
    BRONZE(0.05), SILVER(0.10),  
    GOLD(0.15), PLATINUM(0.25);  
}
```

```
private final double discountRate;
Tier(double discountRate) { /* implement */ }
public double applyDiscount(double price) { /* implement */ }
}
```

Python Function Signature

```
class Tier(Enum):
    BRONZE = 0.05
    SILVER = 0.10
    GOLD = 0.15
    PLATINUM = 0.25

    def apply_discount(self, price: float) -> float: ...
```

Input

A price and a Tier.

Output

The price after discount.

Constraints

Prices are non-negative; discount is applied directly without rounding in this exercise.

Examples

- `Tier.SILVER.applyDiscount(100.0)` → 90.0.
- `Tier.PLATINUM.applyDiscount(200.0)` → 150.0.
- `Tier.BRONZE.applyDiscount(50.0)` → 47.5.

Requirements

- Must use the enum's discount field, not hardcode rates in the method.

Enum Features Practiced

Java: numeric enum field, instance method. **Python:** numeric enum value, instance method.

Learning Objectives

Combine fields and instance methods for data-driven business logic.

Hints

Apply the discount as `price * (1.0 - discountRate)`.

Follow-ups

1. Add a cap on discount amount (e.g. maximum \$50 off) and handle it in the method.

Interview Notes

Simple, but frequently asked because it's a real business requirement; interviewers note whether hardcoding is avoided.

Phase 4 Summary

What was learned: abstract enum methods with constant-specific implementations (Java), instance methods accessing and operating on fields, and algorithms driven by enum data.

Common mistakes: hardcoding formulas, conversion rates, or validation logic outside the enum when it belongs inside; using `ordinal()` in method bodies when safer approaches exist.

Java vs Python differences: Java's constant-specific class body allows anonymous subclass overrides; Python dispatches within shared method bodies or uses conditional logic.

Interview tips: emphasize that enums host behavior, not just data; encapsulation of logic with its constants is a strength.

Mini review: write an abstract method enum with constant-specific implementations (Java) or method dispatch (Python) without notes.

Chapter 5

Phase 5 – Constant Specific Behavior

This phase deepens the per-constant behavior pattern introduced in Q23, showing how enums can enforce wildly different behavior per constant at the type level.

Q29: Animal Sounds

Difficulty: Easy **Phase:** 5 **Estimated Time:** 20 min
Concepts Covered: constant-specific class body, polymorphism

Problem Statement

Each animal has a characteristic sound. Rather than a giant switch, define an `Animal` enum where each constant knows its own sound.

Task

Define an `Animal` enum (`DOG`, `CAT`, `COW`, `DUCK`, `LION`) with an abstract method `sound()` -> `String`. Each constant implements it with its unique sound.

Java Function Signature

```
public enum Animal {  
    DOG { public String sound() { return "Woof"; } },  
    CAT { public String sound() { return "Meow"; } },  
    // ... etc  
  
    abstract public String sound();  
}
```

Python Function Signature

```
class Animal(Enum):  
    DOG = "woof"  
    CAT = "meow"  
    COW = "moo"  
    DUCK = "quack"  
    LION = "roar"  
  
    def sound(self) -> str: ...
```

Input

An `Animal` value.

Output

That animal's sound as a string.

Constraints

Five constants; each must produce a distinct sound.

Examples

- `Animal.DOG.sound()` → "Woof".
- `Animal.LION.sound()` → "Roar".
- Calling on all constants from `values()` produces all five sounds.

Requirements

- Java: must use constant-specific class body; no switch statement in a shared implementation.
- Python: method dispatch dispatching to the right value.

Enum Features Practiced

Java: constant-specific class body, abstract method. **Python:** enum value-driven dispatch.

Learning Objectives

See constant-specific implementations in a purely data-driven context.

Hints

The abstract method forces each constant to provide an implementation at compile time (Java), preventing accidental omissions.

Follow-ups

1. Add a `volume()` method returning an enum (`LOUD`, `SOFT`).

Interview Notes

Tests whether the candidate finds the pattern intuitive or uncomfortable, and whether they'd reach for it naturally or only if prompted.

Q30: Tax Calculator

Difficulty: Medium **Phase:** 5 **Estimated Time:** 25 min
Concepts Covered: constant-specific behavior, fields, algorithms

Problem Statement

Tax rates vary by jurisdiction: some places apply sales tax per item, others apply it only to the subtotal, and others have region-specific rules. Encode the calculation logic in the enum.

Task

Define a `TaxJurisdiction` enum (`SIMPLE`, `PROGRESSIVE`, `EXEMPTED`) with per-constant tax calculation rules. Implement an abstract `calculateTax(double subtotal) -> double` method.

Java Function Signature

```
public enum TaxJurisdiction {
    SIMPLE(0.07) { /* implement calculateTax */ },
    PROGRESSIVE(0.00) { /* implement calculateTax */ },
    EXEMPTED(0.00) { /* implement calculateTax */ };

    private final double baseRate;
    TaxJurisdiction(double baseRate) { this.baseRate = baseRate; }
    abstract double calculateTax(double subtotal);
}
```

Python Function Signature

```
class TaxJurisdiction(Enum):
    SIMPLE = 0.07
    PROGRESSIVE = 0.00
    EXEMPTED = 0.00

    def __init__(self, rate: float): ...
    def calculate_tax(self, subtotal: float) -> float: ...
```

Input

A subtotal amount and a `TaxJurisdiction`.

Output

The tax amount owed.

Constraints

`SIMPLE`: flat 7% tax. `PROGRESSIVE`: 5% up to \$100, 10% above. `EXEMPTED`: no tax.

Examples

- `SIMPLE.calculateTax(100) → 7.0`.
- `PROGRESSIVE.calculateTax(50) → 2.5` (5% of 50).
- `PROGRESSIVE.calculateTax(200) → 15.0` (5% of 100, 10% of 100).
- `EXEMPTED.calculateTax(1000) → 0.0`.

Requirements

- Must use an abstract method with per-constant implementation.
- Each constant must encode its own rule entirely.

Enum Features Practiced

Java: constant-specific implementations with field access. **Python:** method logic accessing enum attributes.

Learning Objectives

Combine fields with per-constant logic to model different algorithms per variant.

Hints

Each constant-specific block in Java can access `this.baseRate` and implement complex logic.

Follow-ups

1. Add a field for the jurisdiction name and use it in error messages.

Interview Notes

A realistic problem where per-constant implementation shines; interviewers watch for whether the candidate sees it as the natural solution or an awkward alternative to hardcoding a switch.

Q31: Chess Piece Movement

Difficulty: **Hard** **Phase:** 5 **Estimated Time:** 40 min
Concepts Covered: constant-specific behavior, complex logic

Problem Statement

Each chess piece (PAWN, KNIGHT, BISHOP, ROOK, QUEEN, KING) moves in a different way. Rather than a monolithic move validator, encode movement rules per piece.

Task

Define a `Piece` enum with an abstract method `isValidMove(from, to, board) -> boolean`. Each piece implements its own movement rules. Assume a standard 8×8 board and no special moves for this exercise.

Java Function Signature

```

public enum Piece {
    PAWN { /* implement isValidMove */ },
    KNIGHT { /* implement isValidMove */ },
    BISHOP { /* implement isValidMove */ },
    // ... etc

    abstract boolean isValidMove(Position from, Position to, Board board
        );
}

public record Position(int row, int col) {}
public record Board(Piece[][] squares) {}

```

Python Function Signature

```

class Piece(Enum):
    PAWN = 1
    KNIGHT = 2
    BISHOP = 3
    ROOK = 4
    QUEEN = 5
    KING = 6

```

```
def is_valid_move(self, from_pos: tuple[int, int],
                  to_pos: tuple[int, int],
                  board: list[list]) -> bool: ...
```

Input

A source position, destination, and the board state.

Output

Boolean, true if the move is valid according to that piece's rules.

Constraints

No castling, en passant, or pawn promotion; ignore piece color (assume no self-capture checks); no move-into-check validation.

Examples

- PAWN from (6, 4) to (5, 4) with empty squares → true (one step forward).
- KNIGHT from (0, 0) to (2, 1) → true (L-shape).
- BISHOP from (0, 0) to (3, 3) with all empty → true (diagonal, clear path).
- ROOK from (0, 0) to (0, 7) with all empty → true (straight, clear path).
- QUEEN from (4, 4) to (6, 6) → true (combines bishop and rook).
- KING from (4, 4) to (5, 5) → true (one square in any direction).
- PAWN from (6, 4) to (5, 3) with no target piece → false (no capture in this example).

Requirements

- Each piece must fully validate its move without calling a shared validator.
- Must check that the destination is on the board.

Enum Features Practiced

Java: constant-specific complex logic. **Python:** method dispatch with board state.

Learning Objectives

Encode non-trivial, per-constant algorithms that differ wildly in structure.

Hints

Factor out helper functions for checking clear paths (diagonal, straight) that multiple pieces reuse, placing them outside the enum or as shared static methods.

Follow-ups

1. Add piece colors (WHITE, BLACK) and prevent self-capture.
2. Add promotion logic for pawns reaching the back rank.
3. Add castling and en passant.

Interview Notes

A challenging problem that separates strong candidates from the rest; shows whether the candidate can design large per-constant implementations cleanly.

Q32: Compression Algorithms

Difficulty: Medium **Phase:** 5 **Estimated Time:** 30 min

Concepts Covered: constant-specific behavior, I/O abstraction

Problem Statement

Different compression algorithms (GZIP, BZIP2, DEFLATE) have different APIs and tradeoffs. Wrap them in an enum to provide a uniform interface.

Task

Define a `CompressionAlgorithm` enum (GZIP, BZIP2, DEFLATE) with abstract methods `compress(byte[]) -> byte[]` and `decompress(byte[]) -> byte[]`. Each constant uses a different underlying library (simulate with different methods).

Java Function Signature

```

public enum CompressionAlgorithm {
    GZIP { /* implement compress/decompress using GZIP */ },
    BZIP2 { /* implement compress/decompress using BZIP2 */ },
    DEFLATE { /* implement compress/decompress using DEFLATE */ };

    abstract byte[] compress(byte[] data) throws IOException;
    abstract byte[] decompress(byte[] data) throws IOException;
}

```

Python Function Signature

```

class CompressionAlgorithm(Enum):
    GZIP = "gzip"
    BZIP2 = "bzip2"
    DEFLATE = "deflate"

    def compress(self, data: bytes) -> bytes: ...
    def decompress(self, data: bytes) -> bytes: ...

```

Input

Byte data and a `CompressionAlgorithm`.

Output

Compressed or decompressed byte data.

Constraints

Use standard library compression (Java `java.util.zip`, Python `zlib/bz2`).

Examples

- `GZIP.compress(data)` produces GZIP-formatted bytes.
- `GZIP.decompress(gzip_bytes)` recovers the original data. Decompressing after compressing with the same algorithm.

Requirements

- Abstract methods must throw (Java) or raise (Python) on error, not silently fail.
- No conditional logic (switch) dispatching to algorithms; each constant implements fully.

Enum Features Practiced

Java: constant-specific I/O logic with exception handling. **Python:** algorithm dispatch using enum members.

Learning Objectives

Use enums to abstract over different library APIs under a single interface.

Hints

Java's `java.util.zip` provides `GZIPInputStream/GZIPOutputStream` and `DeflaterInputStream/DeflaterOutputStream`. Python has `zlib` and `bz2` modules.

Follow-ups

1. Add a `compressionRatio()` method returning an estimate.
2. Benchmark each algorithm on different data sizes.

Interview Notes

Shows whether the constant-specific pattern generalizes to wrapping third-party libraries, a real-world use case.

Q33: Notification Channels

Difficulty: **Medium** **Phase:** 5 **Estimated Time:** 25 min
Concepts Covered: constant-specific behavior, side effects

Problem Statement

Send notifications via different channels (EMAIL, SMS, PUSH). Each channel has a different API and error handling. Wrap them in an enum for type-safe dispatch.

Task

Define a `NotificationChannel` enum (EMAIL, SMS, PUSH) with an abstract method `send(user, message) -> void/None`. Each constant dispatches to the appropriate service.

Java Function Signature

```
public enum NotificationChannel {
    EMAIL { void send(User user, String msg) { /* email API */ } },
    SMS { void send(User user, String msg) { /* SMS API */ } },
    PUSH { void send(User user, String msg) { /* push API */ } };

    abstract void send(User user, String message) throws Exception;
}
```

Python Function Signature

```
class NotificationChannel(Enum):
    EMAIL = "email"
    SMS = "sms"
    PUSH = "push"

    def send(self, user: str, message: str) -> None: ...
```

Input

A user identifier (or user object), a message string, and a channel.

Output

Side effect: message sent via that channel; return success or raise on failure.

Constraints

Simulate the APIs (print to stdout or log); no real external calls needed.

Examples

- `EMAIL.send(user, "Hello")` logs email-send action.
- `SMS.send(user, "Code: 1234")` logs SMS-send action.
- Errors (invalid user, rate limit) are captured in exception handling.

Requirements

- Each channel must handle its own error conditions.

Enum Features Practiced

Java: constant-specific methods with side effects. **Python:** enum-based dispatch for side effects.

Learning Objectives

Use enums for safe dispatch of side effects without a monolithic dispatcher.

Hints

Mock the underlying APIs with log statements; the pattern is what matters, not real integration.

Follow-ups

1. Add retry logic per channel (different retry counts / backoff strategies).
2. Add a `supportsRichText()` method per channel.

Interview Notes

Checks whether the candidate understands enums can drive side effects, not just return values.

Q34: Payment Methods

Difficulty: **Medium** **Phase:** 5 **Estimated Time:** 30 min
Concepts Covered: constant-specific behavior, validation

Problem Statement

Process payments through different methods (CREDIT_CARD, PAYPAL, BANK_TRANSFER). Each has different validation, fee structures, and processing logic.

Task

Define a `PaymentMethod` enum with abstract methods `validate()` and `processPayment(amount)` -> `TransactionResult`. Each constant implements both.

Java Function Signature

```
public enum PaymentMethod {
    CREDIT_CARD { /* implement */ },
    PAYPAL { /* implement */ },
    BANK_TRANSFER { /* implement */ };

    abstract boolean validate(PaymentData data);
    abstract TransactionResult processPayment(PaymentData data, double
        amount);
}
```

Python Function Signature

```
class PaymentMethod(Enum):
    CREDIT_CARD = "cc"
    PAYPAL = "paypal"
    BANK_TRANSFER = "bank"

    def validate(self, data: dict) -> bool: ...
    def process_payment(self, data: dict, amount: float) -> dict: ...
```

Input

Payment data (a dict/object with details) and an amount.

Output

A result object indicating success/failure and fee applied.

Constraints

CREDIT_CARD: 2.9% fee, requires card number. PAYPAL: 3.5% fee, requires email. BANK_TRANSFER: 1% fee, requires account number.

Examples

- CREDIT_CARD.validate(data with valid card) → true.
- CREDIT_CARD.processPayment(\$100) → \$102.90 charged (2.9% fee).
- PAYPAL.processPayment(\$100) → \$103.50 charged.
- BANK_TRANSFER.processPayment(\$100) → \$101.00 charged.

Requirements

- Each payment method must apply its fee.
- Validation must reject incomplete/invalid data.

Enum Features Practiced

Java: multiple abstract methods, per-constant complex implementations. **Python:** multiple method dispatch.

Learning Objectives

Combine validation and processing logic per variant, a pattern seen in real payment gateways.

Hints

Store the fee rate in the enum or compute it per-constant; use a helper method to apply fees.

Follow-ups

1. Add refund support per payment method.
2. Add dispute resolution logic.

Interview Notes

A realistic, complex problem involving multiple methods and conditional logic per constant; tests design ability under realistic constraints.

Phase 5 Summary

What was learned: constant-specific class bodies (Java) as a way to attach different implementations to each constant, independent of shared code; Python's approach via method dispatch; and how to combine fields, validation, and complex logic per constant.

Common mistakes: treating constant-specific behavior as a rare pattern when it's a direct application of polymorphism; writing a single implementation with a massive switch instead of per-constant implementations.

Java vs Python differences: Java's anonymous subclass syntax is verbose but enforces all constants implement abstract methods at compile time; Python uses dispatch within shared methods, more flexible but less type-safe.

Interview tips: position constant-specific behavior as a cleaner, more maintainable alternative to switch statements when behavior varies widely per variant.

Mini review: write a small constant-specific enum (Java) or dispatch-based enum (Python) with multiple methods without notes.

Chapter 6

Phase 6 – Interfaces & Abstract Behavior

This phase shows how enums implement interfaces, extending polymorphism beyond the enum type itself.

Q35: Shape Interface

Difficulty: Medium **Phase:** 6 **Estimated Time:** 25 min
Concepts Covered: enum implementing interface, polymorphic collections

Problem Statement

A graphics library represents shapes (CIRCLE, SQUARE, TRIANGLE) with different area and perimeter calculations. Use an interface to define the contract and let enum constants implement it.

Task

Define a **Shape** interface with **area()** and **perimeter()** methods. Implement it via an enum of shape types, each with appropriate calculations.

Java Function Signature

```
public interface Shape {
    double area();
    double perimeter();
}

public enum ShapeType implements Shape {
    CIRCLE(5) { /* implement */ },
    SQUARE(4) { /* implement */ },
    TRIANGLE(3, 4, 5) { /* implement */ };
    // ... fields and implementations
}
```

Python Function Signature

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
```

```

def area(self) -> float: ...
@abstractmethod
def perimeter(self) -> float: ...

class ShapeType(Shape, Enum):
    CIRCLE = 5
    SQUARE = 4
    TRIANGLE = (3, 4, 5)
    # ... __init__ and implementations

```

Input

A shape type and necessary dimensions (stored in the enum constant).

Output

Computed area and perimeter.

Constraints

CIRCLE radius=5, SQUARE side=4, TRIANGLE sides=3,4,5.

Examples

- `ShapeType.CIRCLE.area()` → $25\pi \approx 78.54$.
- `ShapeType.SQUARE.perimeter()` → 16.0.
- `ShapeType.TRIANGLE.perimeter()` → 12.0.

Requirements

- Must implement the interface, not just provide ad-hoc methods.

Enum Features Practiced

Java: enum implementing interface, constant-specific implementations. **Python:** multiple inheritance with Enum and ABC.

Learning Objectives

Understand enums as implementers of interfaces, enabling polymorphic collections.

Hints

Python requires careful MRO (Method Resolution Order); put Enum before ABC in the inheritance list.

Follow-ups

1. Collect all shapes in a list and sum their areas.

Interview Notes

Tests whether the candidate sees enums as full class participants in OOP hierarchies.

Q36: Vehicle Interface

Difficulty: Medium **Phase:** 6 **Estimated Time:** 25 min
Concepts Covered: interface with fields, enum implementation

Problem Statement

Different vehicles (CAR, MOTORCYCLE, TRUCK) have different acceleration rates, max speeds, and fuel capacities. Implement a **Vehicle** interface, backing it with an enum.

Task

Define a **Vehicle** interface with methods `accelerate()`, `maxSpeed()`, `fuelCapacity()`. Implement via enum with appropriate constants.

Java Function Signature

```
public interface Vehicle {
    double accelerate();
    int maxSpeed();
    int fuelCapacity();
}

public enum VehicleType implements Vehicle {
    CAR(50, 120, 60),
    MOTORCYCLE(40, 140, 15),
    TRUCK(30, 100, 100);

    private final int acceleration, maxSpeed, fuelCapacity;
    VehicleType(int acceleration, int maxSpeed, int fuelCapacity) { /*
        ...*/ }
    // ... interface implementations
}
```

Python Function Signature

```
class Vehicle(ABC):
    @abstractmethod
    def accelerate(self) -> float: ...
    @abstractmethod
    def max_speed(self) -> int: ...
    @abstractmethod
    def fuel_capacity(self) -> int: ...

class VehicleType(Vehicle, Enum):
    CAR = (50, 120, 60)
    MOTORCYCLE = (40, 140, 15)
    TRUCK = (30, 100, 100)
    # ... __init__ and implementations
```

Input

A vehicle type.

Output

Performance metrics.

Constraints

Three vehicles with realistic parameters.

Examples

- `VehicleType.MOTORCYCLE.maxSpeed()` → 140.
- `VehicleType.TRUCK.fuelCapacity()` → 100.

Requirements

- Must store and return interface-specified values from enum fields.

Enum Features Practiced

Java: enum field, interface implementation. **Python:** enum attribute, ABC method implementation.

Learning Objectives

Combine enum fields with interface implementation cleanly.

Hints

No hints needed – straightforward extension of earlier patterns.

Follow-ups

1. Add a method computing range on a full tank.

Interview Notes

Simple, but confirms the candidate understands enums fit into standard OOP patterns.

Q37: Payment Interface

Difficulty: **Medium** **Phase:** 6 **Estimated Time:** 30 min

Concepts Covered: interface defining contract, enum implementation, extensibility

Problem Statement

Extend Q34 (Payment Methods) using an interface. Define a **Processor** interface representing payment processors; have a **PaymentMethod** enum implement it.

Task

Define a **PaymentProcessor** interface with `canProcess()`, `validate()`, `charge()`. Implement via enum.

Java Function Signature

```

public interface PaymentProcessor {
    boolean canProcess(PaymentData data);
    boolean validate(PaymentData data);
    TransactionResult charge(PaymentData data, double amount);
}

public enum PaymentMethod implements PaymentProcessor {

```

```

CREDIT_CARD { /* implement */ },
PAYPAL { /* implement */ },
BANK_TRANSFER { /* implement */ };

// ... interface implementations
}

```

Python Function Signature

```

class PaymentProcessor(ABC):
    @abstractmethod
    def can_process(self, data: dict) -> bool: ...
    @abstractmethod
    def validate(self, data: dict) -> bool: ...
    @abstractmethod
    def charge(self, data: dict, amount: float) -> dict: ...

class PaymentMethod(PaymentProcessor, Enum):
    CREDIT_CARD = "cc"
    PAYPAL = "paypal"
    BANK_TRANSFER = "bank"
    # ... implementations

```

Input

Payment data and amount.

Output

Transaction result.

Constraints

As in Q34; fees apply per method.

Examples

- `PaymentMethod.CREDIT_CARD.charge(data, 100)` → success with fee.

Requirements

- Must implement all interface methods; no fallback to a shared implementation.

Enum Features Practiced

Java: enum implementing concrete interface. **Python:** enum implementing ABC.

Learning Objectives

Enums as implementers of interfaces enables polymorphic dispatch without knowing the specific type.

Hints

A caller can work with `PaymentProcessor` `processor = payment_method`; and call interface methods polymorphically.

Follow-ups

1. Add a factory method: `forData(PaymentData)` returning the appropriate processor.

Interview Notes

Tests deep OOP understanding: enums as polymorphic participants, not special cases.

Q38: Logger

Difficulty: Medium **Phase:** 6 **Estimated Time:** 25 min
Concepts Covered: interface with default methods, enum variants

Problem Statement

A logging system supports multiple outputs (CONSOLE, FILE, NETWORK). Implement a `Logger` interface with a default method for common logic and enum-specific implementations for different outputs.

Task

Define a `Logger` interface with `log(level, message)`. Implement via enum, with one constant per output type. CONSOLE logs to stdout; FILE appends to a test file; NETWORK sends over a mock socket.

Java Function Signature

```
public interface Logger {
    void log(LogLevel level, String message);

    default String formatMessage(LogLevel level, String msg) {
        return String.format("[%s] %s", level, msg);
    }
}

public enum LoggerType implements Logger {
    CONSOLE { /* implement */ },
    FILE { /* implement */ },
    NETWORK { /* implement */ };
}
```

Python Function Signature

```
class Logger(ABC):
    @abstractmethod
    def log(self, level: str, message: str) -> None: ...

    def format_message(self, level: str, msg: str) -> str:
        return f"[{level}] {msg}"

class LoggerType(Logger, Enum):
    CONSOLE = "console"
    FILE = "file"
    NETWORK = "network"
    # ... implementations
```

Input

A log level (DEBUG, INFO, WARN, ERROR) and message.

Output

Side effect: message logged to appropriate destination.

Constraints

Simulate file/network I/O (no real files or sockets required).

Examples

- `LoggerType.CONSOLE.log(DEBUG, "test")` outputs to stdout.
- `LoggerType.FILE.log(INFO, "test")` appends to a test file.

Requirements

- Must use the interface's default method for message formatting.

Enum Features Practiced

Java: enum implementing interface with default methods. **Python:** enum implementing ABC with concrete helper methods.

Learning Objectives

Interfaces provide contracts; default/concrete methods reduce boilerplate in enum implementations.

Hints

Each enum constant can call `formatMessage(...)` to avoid repeating formatting logic.

Follow-ups

1. Add a `shouldLog(level)` method to filter by level.

Interview Notes

Tests whether the candidate leverages default methods to reduce enum implementation boilerplate.

Q39: Database Connectors

Difficulty: **Hard** **Phase:** 6 **Estimated Time:** 35 min
Concepts Covered: interface, multiple methods, enum variants

Problem Statement

An application connects to different databases (POSTGRESQL, MYSQL, SQLITE) via different driver APIs. Implement a `Database` interface; use an enum to wrap different connection logic.

Task

Define a `Database` interface with `connect()`, `query()`, `disconnect()`. Implement via enum, simulating each database's connection and query APIs.

Java Function Signature

```

public interface Database {
    Connection connect(String host, int port, String dbName);
    ResultSet query(Connection conn, String sql);
    void disconnect(Connection conn);
}

public enum DatabaseType implements Database {
    POSTGRESQL { /* PostgreSQL driver simulation */ },
    MYSQL { /* MySQL driver simulation */ },
    SQLITE { /* SQLite driver simulation */ };
}

```

Python Function Signature

```

class Database(ABC):
    @abstractmethod
    def connect(self, host: str, port: int, db_name: str) -> dict: ...
    @abstractmethod
    def query(self, conn: dict, sql: str) -> list: ...
    @abstractmethod
    def disconnect(self, conn: dict) -> None: ...

class DatabaseType(Database, Enum):
    POSTGRESQL = "postgres"
    MYSQL = "mysql"
    SQLITE = "sqlite"
    # ... implementations

```

Input

Connection parameters, SQL query.

Output

Connection object (simulated) and query result (simulated).

Constraints

Simulate driver APIs; no real database connections required. Each database has slightly different parameter formats, error handling, and query result styles.

Examples

- `DatabaseType.POSTGRESQL.connect("localhost", 5432, "mydb")` returns a simulated connection.
- `DatabaseType.MYSQL.query(conn, "SELECT * FROM users")` returns simulated rows.
- Each database may handle SQL dialects slightly differently (simulated).

Requirements

- Each database type must implement all interface methods.
- Connection and result objects can be simulated (dicts, mock objects, etc.).

Enum Features Practiced

Java: enum implementing complex interface. **Python:** enum implementing ABC with stateful operations.

Learning Objectives

Enums can abstract over different third-party libraries, providing a unified interface.

Hints

Mock connection objects using nested classes or dicts; simulate query results as strings or lists.

Follow-ups

1. Add transaction support.
2. Add a prepared statement interface.

Interview Notes

A realistic scenario; tests whether the candidate reaches for enums to unify disparate APIs.

Phase 6 Summary

What was learned: enums implementing interfaces, combining constant-specific behavior with interface contracts, and polymorphic dispatch over enum implementations.

Common mistakes: forgetting that enums are classes and can implement interfaces; failing to implement all interface methods in constant-specific bodies.

Java vs Python differences: Java interface syntax is familiar; Python requires ABC and careful MRO. Java default methods reduce boilerplate; Python has concrete methods on ABC.

Interview tips: emphasize that enums fit naturally into OOP hierarchies and can implement multiple interfaces (Java) or inherit from multiple ABCs (Python with care).

Mini review: write a small interface and an enum implementing it with 2-3 constants without notes.

Chapter 7

Final Remarks

Progress Tracker

Track your completion below. Mark each exercise as complete only when you have written, tested, and verified both the Java and Python implementations.

Q	Exercise	Done
Q1–Q6	Phase 1: Basic Enums	
Q7–Q14	Phase 2: Built-in Methods	
Q15–Q22	Phase 3: Fields & Constructors	
Q23–Q28	Phase 4: Enum Methods	
Q29–Q34	Phase 5: Constant Specific Behavior	
Q35–Q39	Phase 6: Interfaces	

Feature Checklist

By the end of this workbook, you should have hands-on experience with all these features:

Java

- ☐ Enum declaration with constants
- ☐ `values()`, `valueOf()`, `name()`, `ordinal()`
- ☐ `compareTo()`, `equals()`, `toString()` override
- ☐ Switch over enums (pattern matching in Java 17+)
- ☐ Constructors and instance fields
- ☐ Instance methods and static methods
- ☐ Abstract methods with constant-specific class bodies
- ☐ Implementing interfaces
- ☐ `EnumSet` and `EnumMap`
- ☐ Serialization behavior
- ☐ Enum singleton pattern
- ☐ Generic methods over enum types
- ☐ Reverse lookups and caching patterns

Python

- ☐ `Enum` class declaration

- ☐ IntEnum, StrEnum, Flag, IntFlag
- ☐ auto() for automatic values
- ☐ .name and .value
- ☐ __init__, __str__ overrides
- ☐ _missing_() hook for custom resolution
- ☐ @unique decorator
- ☐ Aliases and duplicate values
- ☐ Instance methods and classmethods
- ☐ Pattern matching with enums (Python 3.10+)
- ☐ Serialization (JSON, pickle)
- ☐ Using enums with sets and dicts
- ☐ Reverse lookups and caching

Java Enum Reference

This is a quick lookup for Java enum syntax and APIs encountered in this workbook.

Declaration Styles

```
// Minimal enum
public enum Status { ACTIVE, INACTIVE }

// With constructor and fields
public enum Role {
    ADMIN(3), USER(1);
    private final int level;
    Role(int level) { this.level = level; }
    public int getLevel() { return level; }
}

// With abstract method (constant-specific bodies)
public enum Operation {
    ADD { public int apply(int a, int b) { return a+b; } },
    MUL { public int apply(int a, int b) { return a*b; } };
    abstract int apply(int a, int b);
}

// Implementing interface
public enum HttpStatus implements Serializable {
    OK(200), NOT_FOUND(404);
    private final int code;
    HttpStatus(int code) { this.code = code; }
}
```

API Reference

Method	Use
<code>values()</code>	Returns array of all constants in declaration order
<code>valueOf(String)</code>	Looks up constant by exact name, throws if not found
<code>name()</code>	Returns the exact declared identifier
<code>ordinal()</code>	Returns zero-based position in declaration order
<code>compareTo(E)</code>	Compares by ordinal, returns < 0 , 0 , or > 0
<code>equals(Object)</code>	Identity comparison (same as <code>==</code>)
<code>toString()</code>	Overridable for display text
<code>hashCode()</code>	Consistent with equals
<code>clone()</code>	Throws <code>CloneNotSupportedException</code>

Python Enum Reference

Quick lookup for Python enum syntax and common patterns.

Declaration Styles

```
# Basic enum
class Status(Enum):
    ACTIVE = 1
    INACTIVE = 2

# Using auto()
class Priority(Enum):
    LOW = auto()
    HIGH = auto()

# IntEnum (values comparable as ints)
class Level(IntEnum):
    TRACE = 0
    DEBUG = 1
    INFO = 2

# StrEnum (values comparable as strings)
class Color(StrEnum):
    RED = "red"
    GREEN = "green"

# With custom __init__
class Planet(Enum):
    EARTH = (5.976e24, 6.378e6)
    MARS = (6.421e23, 3.397e6)

    def __init__(self, mass, radius):
        self.mass = mass
        self.radius = radius

# Flag for bitwise operations
class Permission(Flag):
    READ = auto()
    WRITE = auto()
    EXECUTE = auto()

# With _missing_
```

```
class Grade(Enum):
    A = 90
    @classmethod
    def _missing_(cls, value):
        if value > 90:
            return cls.A
        return None
```

API Reference

Attribute / Method	Use
<code>.name</code>	Exact identifier string
<code>.value</code>	Underlying value (int, str, tuple, etc.)
<code>__init__</code>	Constructor, called once per member
<code>__str__</code>	Overridable for display; affects <code>str()</code> but not <code>repr()</code>
<code>__repr__</code>	Usually not overridden
<code>_missing_()</code>	Classmethod hook for custom lookup fallback
<code>@unique</code>	Class decorator preventing duplicate values

Common Interview Questions About Enums

Java

1. **Why use enums instead of constants?** Enums are type-safe, support iteration, and can host behavior. Constants are just integers that the compiler doesn't distinguish.
2. **Is `==` safe for comparing enums?** Yes, always. Enum constants are singletons; `==` and `.equals()` are identical.
3. **What's the difference between `name()` and `toString()`?** `name()` is final and returns the declared identifier. `toString()` is overridable for display text. Use `name()` for parsing and serialization.
4. **Can enums extend other classes?** No. Enums implicitly extend `java.lang.Enum` and cannot extend other classes. They can implement interfaces.
5. **How do you iterate over all enum constants?** `for (MyEnum e : MyEnum.values()) { ... }`
6. **What is a constant-specific class body?** Each constant can override abstract methods individually, creating an anonymous subclass. `PLUS { implementation }` is an anonymous class.
7. **When would you use `EnumSet` vs `EnumMap`?** `EnumSet` is a high-performance `Set` for enums (bit-vector backed). `EnumMap` is a high-performance `Map` keyed by enum (array-backed).
8. **Are enums serialized by name or ordinal?** By name. During deserialization, the name is looked up, not the ordinal. This is safer across versions.

Python

1. **What's the difference between `Enum`, `IntEnum`, and `StrEnum`?** Enum members are only equal to themselves. `IntEnum` members are equal to ints and orderable. `StrEnum` members are strings.
2. **Why can't I compare plain enum members with `<`?** Plain Enum members are not orderable by language design. Use `IntEnum` if you need ordering.
3. **What does `@unique` do?** It's a class decorator that raises `ValueError` if two members have the same value.

4. **What are enum aliases?** If two members have the same value, the second is an alias of the first. Iterating the class skips aliases, but they are accessible by name.
5. **When would I use Flag?** When you need bitwise operations (`|`, `&`, `^`) to combine members, typically for permission or feature flags.
6. **What is the `_missing_hook`?** A classmethod that gets called when a lookup by value fails. Use it for custom fallback logic (e.g., finding the nearest grade).
7. **How do you serialize enums to JSON?** By default, `json.dumps` will not serialize enums. Use a custom encoder that converts to `.name` or `.value`.
8. **Can I iterate an enum?** Yes. `for e in MyEnum:` iterates canonical members (aliases skipped).

Best Practices

- **Use enums for fixed, small, well-known sets of values.** If the set is large or unknown at compile time, use a class hierarchy or a different approach.
- **Keep enums immutable.** Fields should be `private final` (Java) or computed properties (Python).
- **Separate `name()` from `display`.** Use `toString()/__str__` for user-facing text; keep `name()` for parsing and serialization.
- **Use constant-specific behavior (Java) or `dispatch` (Python) instead of giant switches.** It scales better and is safer against missing cases.
- **Cache reverse lookups.** Build a `Map/dict` of values to constants once, not on every lookup.
- **Implement interfaces to enable polymorphism.** Enums can implement interfaces just like classes.
- **Document field meanings and constraints.** Enums with fields should clarify which fields apply to which constants.
- **Use `EnumSet/EnumMap` (Java) for collections.** They are more efficient than generic collections.
- **Make ordinal changes cautiously.** Code that depends on `ordinal()` or declaration order is fragile. Use explicit fields instead.
- **Test serialization.** If enums are persisted, verify they deserialize correctly after adding new constants.

Anti-Patterns

- **Using `ordinal()` for business meaning.** If the order matters, store it explicitly in a field.
- **Storing mutable collections in enum fields.** If you do, document that mutating the collection affects all copies.
- **Mixing `name()` and `toString()`.** Choose one for parsing; use the other for display only.
- **Huge switch statements over enums.** Use constant-specific implementations or polymorphism instead.
- **Enums for something that should be a strategy class.** If behavior is complex or external plugins are needed, use a class hierarchy.
- **Forgetting to validate enum inputs.** Always wrap `valueOf()` or member lookups in `try/catch` or defensive checks.
- **Enums with optional fields.** If not all constants have a field, question whether the enum design is right.

When NOT to Use Enums

- **Unbounded or frequently changing sets of values.** Enums are compile-time constructs; if your domain changes at runtime, use classes or configuration files.
- **Huge enums (100+ constants).** Enums don't scale well above a few dozen constants; consider a different representation.
- **Complex inheritance hierarchies.** Enums can implement interfaces but cannot extend other classes; if you need an inheritance tree, use classes.
- **Plugin systems requiring external registration.** Enums are closed at compile time; use a registry or strategy pattern.
- **Simple one-off branching.** Not every boolean or ternary choice needs an enum. If there's only one or two decision sites, a variable may suffice.

Recommended Further Reading

Java

- *Effective Java* (3rd Edition) by Joshua Bloch – Items 34–37 cover enums in depth.
- Java Language Specification §8.9 – Authoritative enum syntax and semantics.
- JavaDoc for `java.lang.Enum` – Full API reference.
- “Enum Goodness” by Joshua Bloch – Classic oral history of enum design.

Python

- Python documentation on `enum` module – Comprehensive and clear.
- PEP 366, PEP 422, PEP 435 – Evolution of enum design in Python.
- *Fluent Python* by Luciano Ramalho – Chapter on functional style and metaprogramming (enum is used as example).

Appendix A

Appendix: Solution Outline for Q1

This is not a complete solution, but an outline showing the pattern to follow for all exercises in this workbook.

Q1 Traffic Signal (Outline)

Java Pattern

```
public enum Signal {
    RED, YELLOW, GREEN;

    public static Signal next(Signal current) {
        // Consider: What is the order?
        // RED -> GREEN -> YELLOW -> RED
        // Use switch or array indexing with ordinal
    }
}
```

Python Pattern

```
class Signal(Enum):
    RED = auto()
    YELLOW = auto()
    GREEN = auto()

def next_signal(current: Signal) -> Signal:
    # Consider: Can I use a dict or list to map current -> next?
    # Or can I use ordinal-like indexing?
    pass
```

Outline

1. Define the enum type with three constants.
2. Decide on the order: RED $\rightarrow$ GREEN $\rightarrow$ YELLOW $\rightarrow$ RED.
3. Implement next() to return the correct following signal.
4. Test all three transitions and the wrap-around.

Do not copy this; instead, use it as a structure to guide your own thinking.